

TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI TP. HỒ CHÍ MINH
VIỆN CÔNG NGHỆ THÔNG TIN VÀ ĐIỆN, ĐIỆN TỬ



ĐỒ ÁN MÔN HỌC
HỆ QUẢN TRỊ CƠ SỞ DỮ LIỆU

TÊN ĐỀ TÀI
QUẢN LÝ VẬT TƯ
CHO DOANH NGHIỆP XÂY DỰNG

Tp. Hồ Chí Minh, ngày tháng năm 2026

-- Session 2 BEGIN TRANSACTION;

```
UPDATE TON_KHO SET SoLuongTon = SoLuongTon + 100 WHERE MaKho
= 'KHO01' AND MaVT = 'VT01';
```

```
COMMIT TRANSACTION;
```

```
-- Session 2 BEGIN TRANSACTION;
```

```
UPDATE TON_KHO SET SoLuongTon = SoLuongTon + 100 WHERE MaKho
= 'KHO01' AND MaVT = 'VT01';
```

```
COMMIT TRANSACTION;
```

```
-- Session 2 BEGIN TRANSACTION;
```

```
UPDATE TON_KHO SET SoLuongTon = SoLuongTon + 100 WHERE MaKho
= 'KHO01' AND MaVT = 'VT01';
```

```
COMMIT TRANSACTION;
```

```
-- Session 2 BEGIN TRANSACTION;
```

```
UPDATE TON_KHO SET SoLuongTon = SoLuongTon + 100 WHERE MaKho
= 'KHO01' AND MaVT = 'VT01';
```

```
COMMIT TRANSACTION;
```

```
-- Session 2 BEGIN TRANSACTION;
```

```
UPDATE TON_KHO SET SoLuongTon = SoLuongTon + 100 WHERE MaKho
= 'KHO01' AND MaVT = 'VT01';
```

```
COMMIT TRANSACTION;
```

```
-- Session 1: Sử dụng mức cô lập REPEATABLE READ để giữ dữ liệu đọc ổn
định SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;
```

```
BEGIN TRANSACTION;
```

```
-- Đọc số lượng tồn kho lần đầu SELECT MaKho, MaVT, SoLuongTon FROM
TON_KHO WHERE MaKho = 'KHO01' AND MaVT = 'VT01';
```

```
-- Tạo khoảng thời gian để Session 2 thực hiện UPDATE WAITFOR DELAY
'00:00:08';
```

```
-- Đọc lại dữ liệu để kiểm tra tính nhất quán SELECT MaKho, MaVT,
SoLuongTon FROM TON_KHO WHERE MaKho = 'KHO01' AND MaVT = 'VT01';
```

COMMIT TRANSACTION;

-- Session 2: Thực hiện cập nhật dữ liệu tồn kho -- Sẽ bị BLOCK nếu Session 1 chưa COMMIT BEGIN TRANSACTION;

UPDATE TON_KHO SET SoLuongTon = SoLuongTon + 100 WHERE MaKho = 'KHO01' AND MaVT = 'VT01';

COMMIT TRANSACTION;

-- Session 1: Sử dụng mức cô lập REPEATABLE READ để giữ dữ liệu đọc ổn định SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;

BEGIN TRANSACTION;

-- Đọc số lượng tồn kho lần đầu SELECT MaKho, MaVT, SoLuongTon FROM TON_KHO WHERE MaKho = 'KHO01' AND MaVT = 'VT01';

-- Tạo khoảng thời gian để Session 2 thực hiện UPDATE WAITFOR DELAY '00:00:08';

-- Đọc lại dữ liệu để kiểm tra tính nhất quán SELECT MaKho, MaVT, SoLuongTon FROM TON_KHO WHERE MaKho = 'KHO01' AND MaVT = 'VT01';

COMMIT TRANSACTION;

-- Session 2: Thực hiện cập nhật dữ liệu tồn kho -- Sẽ bị BLOCK nếu Session 1 chưa COMMIT BEGIN TRANSACTION;

UPDATE TON_KHO SET SoLuongTon = SoLuongTon + 100 WHERE MaKho = 'KHO01' AND MaVT = 'VT01';

COMMIT TRANSACTION;

-- Session 1: Sử dụng mức cô lập REPEATABLE READ để giữ dữ liệu đọc ổn định SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;

BEGIN TRANSACTION;

-- Đọc số lượng tồn kho lần đầu SELECT MaKho, MaVT, SoLuongTon FROM TON_KHO WHERE MaKho = 'KHO01' AND MaVT = 'VT01';

-- Tạo khoảng thời gian để Session 2 thực hiện UPDATE WAITFOR DELAY '00:00:08';

```
-- Đọc lại dữ liệu để kiểm tra tính nhất quán SELECT MaKho, MaVT,
SoLuongTon FROM TON_KHO WHERE MaKho = 'KHO01' AND MaVT = 'VT01';
```

```
COMMIT TRANSACTION;
```

```
-- Session 2: Thực hiện cập nhật dữ liệu tồn kho -- Sẽ bị BLOCK nếu Session 1
chưa COMMIT BEGIN TRANSACTION;
```

```
UPDATE TON_KHO SET SoLuongTon = SoLuongTon + 100 WHERE MaKho
= 'KHO01' AND MaVT = 'VT01';
```

```
COMMIT TRANSACTION;
```

```
-- Session 1: Sử dụng mức cô lập REPEATABLE READ để giữ dữ liệu đọc ổn
định SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;
```

```
BEGIN TRANSACTION;
```

```
-- Đọc số lượng tồn kho lần đầu SELECT MaKho, MaVT, SoLuongTon FROM
TON_KHO WHERE MaKho = 'KHO01' AND MaVT = 'VT01';
```

```
-- Tạo khoảng thời gian để Session 2 thực hiện UPDATE WAITFOR DELAY
'00:00:08';
```

```
-- Đọc lại dữ liệu để kiểm tra tính nhất quán SELECT MaKho, MaVT,
SoLuongTon FROM TON_KHO WHERE MaKho = 'KHO01' AND MaVT = 'VT01';
```

```
COMMIT TRANSACTION;
```

```
-- Session 2: Thực hiện cập nhật dữ liệu tồn kho -- Sẽ bị BLOCK nếu Session 1
chưa COMMIT BEGIN TRANSACTION;
```

```
UPDATE TON_KHO SET SoLuongTon = SoLuongTon + 100 WHERE MaKho
= 'KHO01' AND MaVT = 'VT01';
```

```
COMMIT TRANSACTION;
```

TÓM TẮT ĐỒ ÁN

MỤC LỤC

TÓM TẮT ĐỒ ÁN.....	i
MỤC LỤC	ii
DANH MỤC CÁC TỪ VIẾT TẮT	iii
DANH MỤC CÁC HÌNH VẼ	iv
DANH MỤC CÁC BẢNG.....	iv
CHƯƠNG 1: CƠ SỞ LÝ THUYẾT	1
CHƯƠNG 2: KHẢO SÁT THỰC TRẠNG VÀ XÁC ĐỊNH YÊU CẦU	2
CHƯƠNG 3: THIẾT KẾ HỆ THỐNG.....	3
CHƯƠNG 4: XÂY DỰNG VÀ TRIỂN KHAI HỆ THỐNG	4
4.1. Công nghệ sử dụng	4
4.2. Xây dựng cơ sở dữ liệu.....	4
4.3. Xây dựng chức năng đăng nhập và phân quyền	4
CHƯƠNG 5: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	5
5.1. Kiểm thử các chức năng chính	5
5.2. Đánh giá ưu điểm, hạn chế của hệ thống	5
5.3. Kết luận kết quả đạt được	5
5.4. Đề xuất hướng phát triển.....	5
DANH MỤC TÀI LIỆU THAM KHẢO	5

DANH MỤC CÁC TỪ VIẾT TẮT

DANH MỤC CÁC HÌNH VẼ

[No table of contents entries found.](#)

DANH MỤC CÁC BẢNG

No table of contents entries found.

CHƯƠNG 1: PHÂN TÍCH BÀI TOÁN

1.1. Mô tả bài toán quản lý vật tư

Doanh nghiệp xây dựng sử dụng nhiều loại vật tư trong quá trình thi công như xi măng, sắt thép, cát, đá, gạch, thiết bị điện và dụng cụ. Vật tư được mua từ nhà cung cấp, nhập vào kho của doanh nghiệp rồi cấp cho các công trình theo nhu cầu sử dụng.

Khi quản lý bằng sổ sách hoặc nhiều tệp riêng lẻ, số liệu giữa các bộ phận dễ bị chênh lệch. Nhân viên khó xác định chính xác số lượng vật tư tại từng kho, vật tư đã cấp cho công trình nào và số lượng còn lại. Việc kiểm tra lịch sử nhập, xuất, chuyển kho cũng mất nhiều thời gian. Một số trường hợp có thể xảy ra như xuất vượt số lượng tồn, thiếu vật tư phục vụ thi công hoặc số liệu trên hệ thống không khớp với kết quả kiểm kê.

Nhóm xây dựng hệ thống quản lý vật tư để lưu trữ thông tin tập trung và theo dõi các thay đổi của tồn kho. Quy trình quản lý vật tư gồm:

Nhà cung cấp → Nhập vật tư vào kho → Theo dõi tồn kho → Xuất vật tư cho công trình → Chuyển kho hoặc điều chỉnh tồn → Tra cứu và thống kê.

Hệ thống quản lý vật tư thuộc sở hữu của doanh nghiệp xây dựng, từ lúc nhập từ nhà cung cấp đến khi cấp cho các công trình do doanh nghiệp thi công. Hệ thống không quản lý bán vật tư, hóa đơn bán hàng hoặc công nợ khách hàng.

1.2. Mục tiêu và phạm vi hệ thống

1.2.1. Mục tiêu của hệ thống

Hệ thống có các mục tiêu sau:

Quản lý thông tin loại vật tư, vật tư, nhà cung cấp, kho và công trình.

Ghi nhận vật tư được nhập từ nhà cung cấp vào kho.

Ghi nhận vật tư được xuất từ kho cho từng công trình.

Theo dõi việc chuyển vật tư giữa các kho.

Điều chỉnh số lượng tồn khi kết quả kiểm kê có sai lệch.

Theo dõi số lượng của từng vật tư tại từng kho.

Kiểm tra số lượng tồn trước khi xuất hoặc chuyển vật tư.

Lưu người lập, thời gian và nội dung của từng phiếu.

Tra cứu lịch sử nhập, xuất, chuyển và điều chỉnh vật tư.

Cảnh báo vật tư có số lượng thấp hơn mức tồn tối thiểu.

Thông kê nhập – xuất – tồn theo thời gian, kho, vật tư hoặc công trình.

Giữ số liệu phiếu và tồn kho thống nhất khi nghiệp vụ được thực hiện.

1.2.2. Phạm vi chức năng

Phạm vi của hệ thống: Hệ thống tập trung quản lý vật tư nội bộ của doanh nghiệp xây dựng, từ khi vật tư được nhập từ nhà cung cấp, lưu trữ tại các kho, cấp cho công trình, chuyển giữa các kho và điều chỉnh sau kiểm kê. Hệ thống không quản lý hoạt động bán vật tư, hóa đơn bán hàng và công nợ khách hàng.

Hệ thống gồm các chức năng sau:

1. Đăng nhập hệ thống.
2. Quản lý tài khoản người dùng.
3. Quản lý loại vật tư.
4. Quản lý vật tư.
5. Quản lý nhà cung cấp.
6. Quản lý kho.
7. Quản lý công trình.
8. Nhập vật tư từ nhà cung cấp vào kho.
9. Xuất vật tư từ kho cho công trình.
10. Chuyển vật tư giữa các kho.
11. Điều chỉnh số lượng tồn sau kiểm kê.
12. Tra cứu tồn kho theo kho hoặc vật tư.
13. Xem lịch sử nhập, xuất, chuyển và điều chỉnh.
14. Cảnh báo vật tư dưới mức tồn tối thiểu.
15. Thông kê tình hình nhập – xuất – tồn.

1.3. Các đối tượng nghiệp vụ và chức năng quản lý

1.3.1. Các đối tượng nghiệp vụ

Hệ thống quản lý các đối tượng sau:

Bảng 1.1. Các đối tượng nghiệp vụ và vai trò trong hệ thống

STT	Đối tượng	Vai trò của hệ thống
1	Tài khoản	Lưu thông tin người sử dụng và xác định người lập phiếu.
2	Loại vật tư	Phân nhóm và quản lý các loại vật tư trong doanh nghiệp.
3	Vật tư	Lưu thông tin vật tư được nhập, lưu trữ tại kho và cấp cho các công trình.
4	Nhà cung cấp	Cung cấp vật tư cho doanh nghiệp và được ghi nhận trong phiếu nhập.
5	Kho	Quản lý các địa điểm lưu trữ vật tư của doanh nghiệp.
6	Công trình	Quản lý các công trình tiếp nhận và sử dụng vật tư được cấp từ kho.
7	Tồn kho	Theo dõi và lưu số lượng của từng vật tư tại từng kho.
8	Phiếu nhập	Ghi nhận vật tư được nhập từ nhà cung cấp vào kho.
9	Phiếu xuất	Ghi nhận vật tư được cấp từ kho cho công trình.
10	Phiếu chuyển kho	Ghi nhận việc chuyển vật tư từ kho này sang kho khác.
11	Phiếu điều chỉnh	Ghi nhận thay đổi số lượng tồn khi kiểm kê có chênh lệch.

Tài khoản: lưu thông tin người sử dụng và xác định người lập phiếu.

Loại vật tư: dùng để phân nhóm vật tư.

Vật tư: lưu thông tin các vật tư được nhập, lưu kho và cấp cho công trình.

Nhà cung cấp: cung cấp vật tư cho doanh nghiệp và được ghi nhận trong phiếu nhập.

Kho: nơi lưu trữ vật tư của doanh nghiệp.

Công trình: nơi tiếp nhận và sử dụng vật tư được cấp từ kho.

Tồn kho: lưu số lượng của từng vật tư tại từng kho.

Phiếu nhập: ghi nhận vật tư được nhập từ nhà cung cấp vào kho.

Phiếu xuất: ghi nhận vật tư được cấp từ kho cho công trình.

Phiếu chuyển kho: ghi nhận việc chuyển vật tư giữa hai kho.

Phiếu điều chỉnh: ghi nhận thay đổi số lượng tồn khi kiểm kê có chênh lệch.

Mỗi phiếu nghiệp vụ được tách thành phần thông tin chung và phần chi tiết. Thông tin chung lưu ngày lập, người lập, kho và trạng thái phiếu; phần chi tiết lưu danh sách vật tư cùng số lượng tương ứng. Cách tổ chức này cho phép một phiếu chứa nhiều vật tư.

1.3.2. Các chức năng nghiệp vụ chính

Nhập kho: ghi nhận vật tư nhập từ nhà cung cấp và tăng số lượng tồn tại kho nhận theo từng loại vật tư.

Xuất cho công trình: kiểm tra số lượng tồn, lập phiếu xuất và giảm số lượng vật tư tại kho xuất theo số lượng cấp.

Chuyển kho: giảm vật tư tại kho nguồn và tăng vật tư tại kho đích theo số lượng chuyển.

Điều chỉnh tồn: cập nhật số lượng tồn theo kết quả kiểm kê và lưu lý do điều chỉnh của từng vật tư.

Tra cứu tồn kho: xem số lượng vật tư theo kho hoặc theo vật tư.

Cảnh báo tồn thấp: xác định vật tư có số lượng tồn thấp hơn mức tồn tối thiểu được thiết lập trước.

Xem lịch sử: tra cứu các phiếu nhập, xuất, chuyển và điều chỉnh đã lập trong hệ thống.

Thông kê: tổng hợp số liệu nhập – xuất – tồn theo thời gian, kho, vật tư hoặc công trình phục vụ quản lý.

1.4. Các quy tắc nghiệp vụ của hệ thống

Hệ thống áp dụng các quy tắc sau:

Quy tắc về dữ liệu danh mục

- Mỗi tài khoản, loại vật tư, vật tư, nhà cung cấp, kho và công trình có một mã duy nhất.
- Tên đăng nhập của các tài khoản không được trùng nhau.
- Mỗi vật tư phải thuộc một loại vật tư đã có trong hệ thống.
- Danh mục đã phát sinh dữ liệu không bị xóa trực tiếp. Khi không còn sử dụng, trạng thái của danh mục được chuyển sang ngừng hoạt động.

Quy tắc về tồn kho

- Số lượng tồn được quản lý theo từng kho và từng vật tư.
- Mỗi cặp kho và vật tư chỉ có một bản ghi tồn kho.
- Số lượng tồn kho không được nhỏ hơn 0.
- Vật tư có số lượng tồn nhỏ hơn mức tồn tối thiểu được xác định và đưa vào danh sách cảnh báo để người quản lý kịp thời bổ sung.

Quy tắc về phiếu nghiệp vụ

- Một phiếu có thể chứa nhiều vật tư.
- Một vật tư không được xuất hiện nhiều lần trong cùng một phiếu.
- Số lượng nhập, xuất và chuyển phải lớn hơn 0.
- Đơn giá nhập không được nhỏ hơn 0.
- Người lập phiếu phải có tài khoản trong hệ thống.
- Phiếu điều chỉnh phải có lý do và lưu số lượng trước, sau khi điều chỉnh.

Quy tắc nhập, xuất và chuyển kho

- Nhập kho làm tăng số lượng tồn tại kho nhận.
- Xuất kho làm giảm số lượng tồn tại kho xuất.
- Số lượng xuất không được lớn hơn số lượng tồn.
- Chuyển kho làm giảm tồn kho nguồn và tăng tồn kho đích.
- Số lượng chuyển không được lớn hơn số lượng tồn tại kho nguồn.

- Kho nguồn và kho đích phải khác nhau.

Quy tắc đảm bảo toàn vẹn

- Số lượng tồn không được sửa trực tiếp mà được thay đổi thông qua nghiệp vụ nhập, xuất, chuyển hoặc điều chỉnh.
- Phiếu, chi tiết phiếu và tồn kho phải được xử lý trong cùng một giao tác.
- Khi xảy ra lỗi, giao tác phải hủy toàn bộ thay đổi để tránh chênh lệch dữ liệu.
- Ngày kết thúc dự kiến của công trình không được nhỏ hơn ngày bắt đầu.

Từ các đối tượng và quy tắc nghiệp vụ đã xác định, nhóm tiến hành phân tích các thực thể, thuộc tính, mối quan hệ và ràng buộc để xây dựng mô hình cơ sở dữ liệu cho hệ thống. Nội dung này được trình bày trong Chương 2.

CHƯƠNG 2: PHÂN TÍCH VÀ THIẾT KẾ CƠ SỞ DỮ LIỆU

2.1. Xác định các thực thể và thuộc tính

NHA_CUNG_CAP (Nhà cung cấp): MaNCC, TenNCC, DiaChi, SoDienThoai, Email, TrangThai

LOAI_VAT_TU (Loại vật tư): MaLoai, TenLoai, MoTa

VAT_TU (Vật tư): MaVT, TenVT, MaLoai, DonViTinh, TonToiThieu, MoTa, TrangThai

KHO (Kho): MaKho, TenKho, DiaChi, MoTa, TrangThai

CONG_TRINH (Công trình): MaCT, TenCT, DiaDiem, NgayBatDau, NgayKetThucDuKien, TrangThai

TAI_KHOAN (Tài khoản): MaTK, TenDangNhap, MatKhau, HoTen, VaiTro, TrangThai

PHIEU_NHAP (Phiếu nhập): MaPN, MaNCC, MaKho, NgayNhap, NguoiLap, GhiChu, TrangThai

PHIEU_XUAT (Phiếu xuất): MaPX, MaKho, MaCT, NgayXuat, NguoiLap, GhiChu, TrangThai

PHIEU_CHUYEN_KHO (Phiếu chuyển kho): MaPCK, MaKhoNgon, MaKhoDich, NgayChuyen, NguoiLap, GhiChu, TrangThai

PHIEU_DIEU_CHINH (Phiếu điều chỉnh): MaPDC, MaKho, NgayDieuChinh, NguoiLap, LyDo, TrangThai

2.2. Xác định các mqh và ràng buộc trong csdl

NHA_CUNG_CAP (0,n) – CungCap – (1,1) PHIEU_NHAP

TAI_KHOAN (0,n) – LapPN – (1,1) PHIEU_NHAP

TAI_KHOAN (0,n) – LapPX – (1,1) PHIEU_XUAT

TAI_KHOAN (0,n) – LapPDC – (1,1) PHIEU_DIEU_CHINH

TAI_KHOAN (0,n) – LapPCK – (1,1) PHIEU_CHUYEN_KHO

KHO (0,n) – NhapTai – (1,1) PHIEU_NHAP

KHO (0,n) – XuatTai – (1,1) PHIEU_XUAT

KHO (0,n) – DieuChinhTai – (1,1) PHIEU_DIEU_CHINH

KHO (0,n) – LaKhoNgon – (1,1) PHIEU_CHUYEN_KHO

KHO (0,n) – LaKhoDich – (1,1) PHIEU_CHUYEN_KHO

PHIEU_NHAP (1,n) – CTPhieuNhap – (0,n) VAT_TU

PHIEU_XUAT (1,n) – CTPhieuXuat – (0,n) VAT_TU

PHIEU_DIEU_CHINH (1,n) – CTPhieuDieuChinh – (0,n) VAT_TU

PHIEU_CHUYEN_KHO (1,n) – CTPhieuChuyenKho – (0,n) VAT_TU

KHO (0,n) – TonKho – (0,n) VAT_TU

LOAI_VAT_TU (0,n) – PhanLoai – (1,1) VAT_TU

CONG_TRINH (0,n) – NhanVatTu – (1,1) PHIEU_XUAT

Ràng buộc toàn vẹn dữ liệu

Mã của mỗi đối tượng và mỗi phiếu phải là duy nhất; tên đăng nhập không được trùng nhau.

Các khóa ngoại phải tham chiếu đến bản ghi đã tồn tại trong bảng liên quan.

Mỗi phiếu khi hoàn thành phải có ít nhất một dòng chi tiết và một vật tư không được lặp lại trong cùng một phiếu.

Số lượng nhập, xuất và chuyển phải lớn hơn 0; số lượng tồn và mức tồn tối thiểu không được âm.

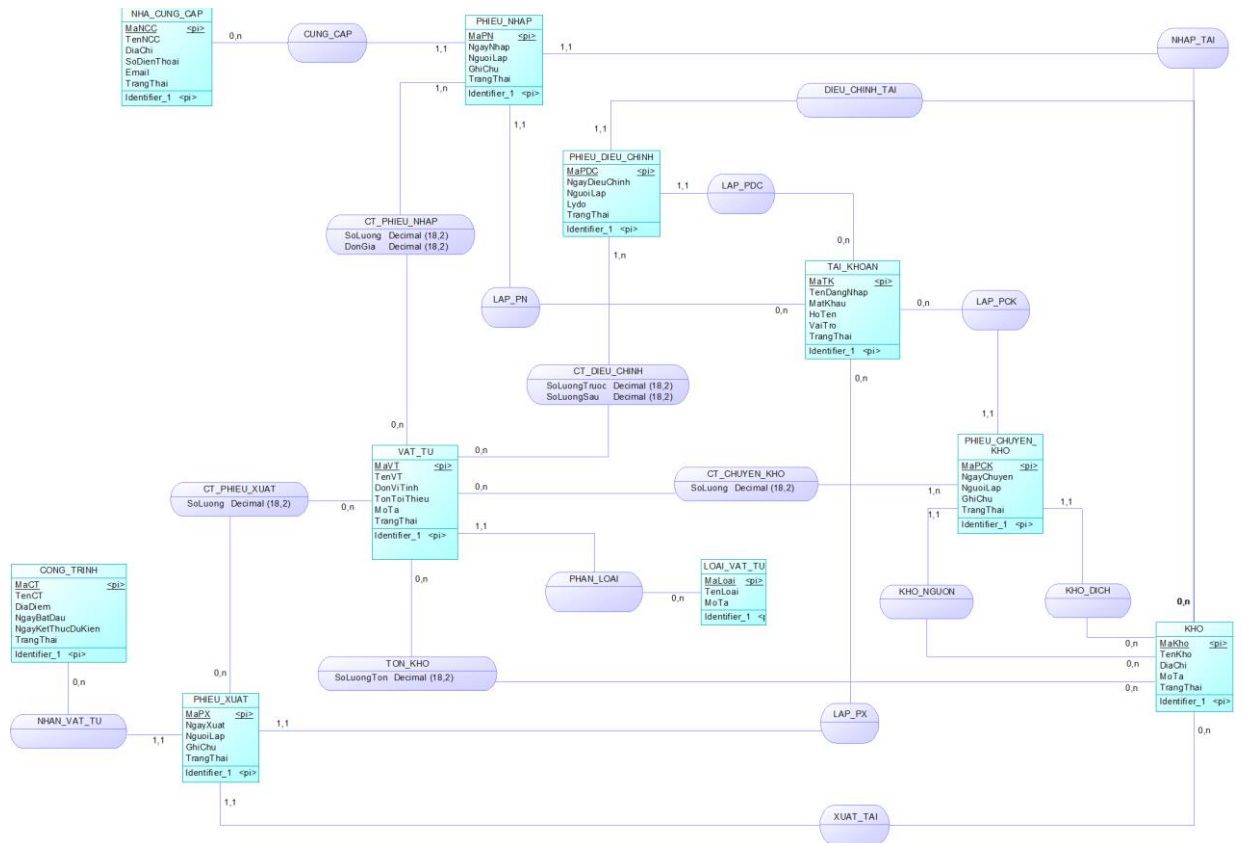
Không được xuất hoặc chuyển vật tư vượt quá số lượng tồn tại kho tương ứng.

Kho nguồn và kho đích trong phiếu chuyển kho phải khác nhau.

Điều chỉnh tồn phải có lý do; số lượng trước và sau điều chỉnh phải khác nhau và không được âm.

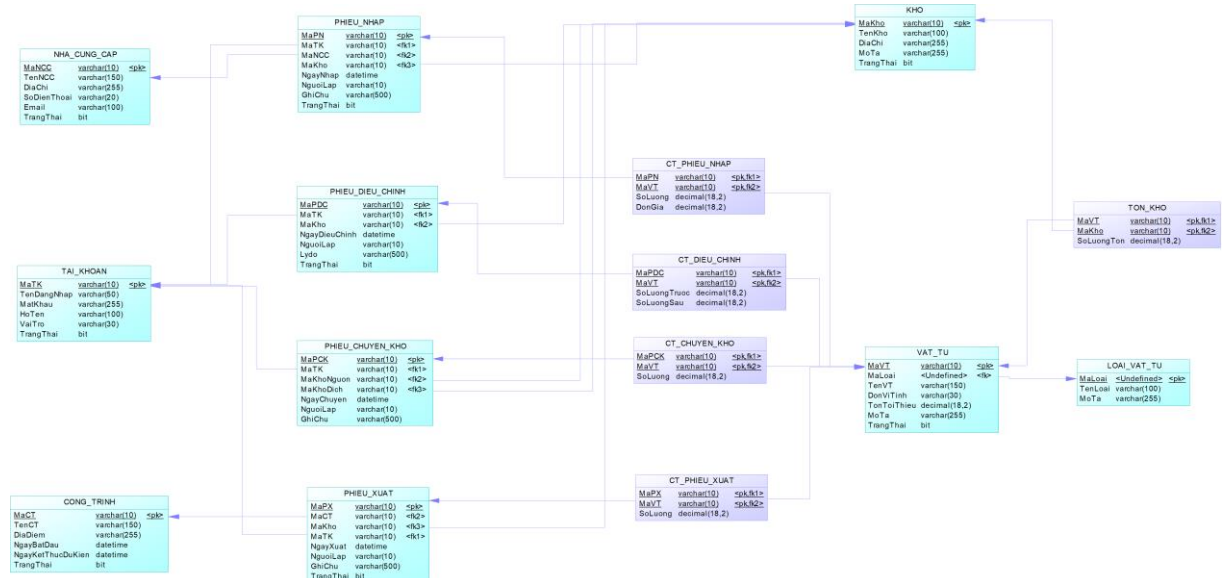
Trạng thái phải thuộc các giá trị đã quy định cho từng nhóm dữ liệu; các nghiệp vụ làm thay đổi tồn kho phải được xử lý bằng transaction và rollback khi xảy ra lỗi.

2.3. Mô hình thực thể - liên kết (ERD)



Hình 2.1. Sơ đồ thực thể – liên kết (ERD) của hệ thống

2.4. Chuyển đổi sang mô hình quan hệ



Hình 2.2. Lược đồ cơ sở dữ liệu của hệ thống

2.5. Thiết kế chi tiết các bảng dữ liệu và ràng buộc

Bảng 2.1. Mô tả bảng TAI_KHOAN

Tên cột	Kiểu dữ liệu	Khóa	NULL	Mặc định	Ràng buộc	Ý nghĩa
MaTK	VARCHAR(10)	PK	Không		Duy nhất	Mã tài khoản
TenDang Nhap	VARCHAR(50)	UNIQUE	Không		Không được trùng	Tên đăng nhập
MatKau	VARCHAR(255)		Không			Mật khẩu của tài khoản
HoTen	NVARCHAR(100)		Không			Họ tên người dùng
VaiTro	VARCHAR(30)		Không		CHECK IN (QUAN_LY, NHAN_VIEN)	Vai trò của người dùng
TrangThai	BIT		Không	1	DEFAULT	1: hoạt động; 0: dừng

Bảng 2.2. Mô tả bảng LOAI_VAT_TU

Tên cột	Kiểu dữ liệu	Khóa	NULL	Mặc định	Ràng buộc	Ý nghĩa
MaLoai	VARCHAR(10)	PK	Không		Duy nhất	Mã loại vật tư
TenLoai	NVARCHAR(100)		Không			Tên loại vật tư
MoTa	NVARCHAR(255)		Có			Mô tả loại vật tư

Bảng 2.3. Mô tả bảng VAT_TU

Tên cột	Kiểu dữ liệu	Khóa	NUL	Mặc định	Ràng buộc	Ý nghĩa
MaVT	VARCHAR(10)	PK	Không		Duy nhất	Mã vật tư
TenVT	NVARCHAR(100)		Không			Tên vật tư
MaLoai	VARCHAR(10)	FK	Không		Tham chiếu LOAI_VAT_TU(MaLoai)	Mã loại của vật tư
DonViTin h	NVARCHAR(30)		Không			Đơn vị tính của vật tư
TonToi Thieu	DECIMAL(18,2)		Không		CHECK ≥ 0	Mức tồn tối thiểu
MoTa	NVARCHAR(255)		Có			Mô tả vật tư
TrangThai	BIT		Không	1	DEFAULT	Trạng thái sử dụng của vật tư

Bảng 2.4. Mô tả bảng NHA_CUNG_CAP

Tên cột	Kiểu dữ liệu	Khóa	NULL	Mặc định	Ràng buộc	Ý nghĩa
MaNCC	VARCHAR(10)	PK	Không		Duy nhất	Mã nhà cung cấp
TenNCC	NVARCHAR(150)		Không			Tên nhà cung cấp
DiaChi	NVARCHAR(255)		Có			Địa chỉ nhà cung cấp
SoDienThoai	VARCHAR(20)		Có			Số điện thoại
Email	VARCHAR(100)		Có			Địa chỉ Email
TrangThai	BIT		Không	1	DEFAULT	Trạng thái hoạt động

Bảng 2.5. Mô tả bảng KHO

Tên cột	Kiểu dữ liệu	Khóa	NULL	Mặc định	Ràng buộc	Ý nghĩa
---------	--------------	------	------	----------	-----------	---------

MaKho	VARCHAR(10)	PK	Không		Duy nhất	Mã kho
TenKho	NVARCHAR(100)		Không			Tên kho
DiaChi	NVARCHAR(255)		Có			Địa chỉ kho
MoTa	NVARCHAR(255)		Có			Mô tả kho
TrangThai	BIT		Không	1	DEFAULT	Trạng thái hoạt động của kho

Bảng 2.6. Mô tả bảng CONG_TRINH

Tên cột	Kiểu dữ liệu	Khóa	NULL	Mặc định	Ràng buộc	Ý nghĩa
MaCT	VARCHAR(10)	PK	Không		Duy nhất	Mã công trình
TenCT	NVARCHAR(150)		Không			Tên công trình
DiaDiem	NVARCHAR(255)		Không			Địa điểm thi công
NgayBatDau	DATE		Có			Ngày

						bắt đầu
NgàyKetThucDuKien	DATE		Có		CHECK (NgàyKetThucDuKien ≥ NgàyBatDau) nếu 2 ngày đều hợp lệ	Ngày kết thúc dự kiến
TrangThai	VARCHAR(30)		Không	'CHUA_BAT_DAU'	CHECK IN (CHUA_BAT_DAU, DANG_THI_CONG, TAM_DUNG, HOAN_THANH)	Trạng thái công trình

Bảng 2.7. Mô tả bảng TON_KHO

Tên cột	Kiểu dữ liệu	Khóa	NULL	Mặc định	Ràng buộc	Ý nghĩa
MaKho	VARCHAR(10)	PK, FK	Không		Tham chiếu KHO(MaKho)	Kho đang lưu vật tư

MaVT	VARCHAR(10)	PK, FK	Không		Tham chiếu VAT_TU(MaVT)	Vật tư được lưu trong kho
SoLuongTon	DECIMAL(18,2)		Không	0	CHECK ≥ 0	Số lượng tồn vật tư tại kho

Bảng 2.8. Mô tả bảng PHIEU_NHAP

Tên cột	Kiểu dữ liệu	Khóa	NULL	Mặc định	Ràng buộc	Ý nghĩa
MaPN	VARCHAR(10)	PK	Không		Duy nhất	Mã phiếu nhập
MaNCC	VARCHAR(10)	FK	Không		Tham chiếu NHA_CUNG_CAP (MaNCC)	Mã nhà cung cấp vật tư
MaKho	VARCHAR(10)	FK	Không		Tham chiếu KHO(MaKho)	Mã kho vật tư
NgayNhap	DATETIME		Không	GETDATE()	DEFAULT	Ngày lập phiếu nhập
NguoiLap	VARCHAR(10)	FK	Không		Tham chiếu TAI_KHOAN (MaTK)	Người lập phiếu
GhiChu	NVARCHAR(500)		Có			Ghi chú của phiếu
TrangThai	VARCHAR(20)		Không	NHAP	CHECK IN (NHAP, HOAN_THANH, DA_HUY)	Trạng thái phiếu nhập

Bảng 2.9. Mô tả bảng CT_PHIEU_NHAP

Tên cột	Kiểu dữ liệu	Khóa	NULL	Mặc định	Ràng buộc	Ý nghĩa
MaPN	VARCHAR(10)	PK, FK	Không		Tham chiếu PHIEU_NHAP (MaPN)	Mã phiếu nhập
MaVT	VARCHAR(10)	PK, FK	Không		Tham chiếu VAT_TU(MaVT)	Mã vật tư
SoLuong	DECIMAL(18,2)		Không		CHECK > 0	Số lượng vật tư nhập
DonGia	DECIMAL(18,2)		Không		CHECK > 0	Đơn giá nhập vật tư

Bảng 2.10. Mô tả bảng PHIEU_XUAT

Tên cột	Kiểu dữ liệu	Khóa	NUL L	Mặc định	Ràng buộc	Ý nghĩa
---------	--------------	------	----------	----------	-----------	------------

MaPX	VARCHAR(10)	PK	Không		Duy nhất	Mã phiếu xuất
MaKho	VARCHAR(10)	FK	Không		Tham chiếu KHO(MaKho)	Mã kho xuất vật tư
MaCT	VARCHAR(10)	FK	Không		Tham chiếu CONG_TRINH (MaCT)	Mã công trình nhận vật tư
NgàyXuất	DATETIME		Không	GETDATE()	DEFAULT	Ngày lập phiếu xuất
Nguoilập	VARCHAR(10)	FK	Không		Tham chiếu TAI_KHOAN (MaTK)	Người lập phiếu
GhiChu	NVARCHAR(500)		Có			Ghi chú của phiếu
TrangThái	VARCHAR(20)		Không	NHAP	CHECK IN (NHAP, HOAN_THAN H, DA_HUY)	Trạng thái phiếu xuất

Bảng 2.11. Mô tả bảng CT_PHIEU_XUAT

Tên cột	Kiểu dữ liệu	Khóa	NULL	Mặc định	Ràng buộc	Ý nghĩa
MaPX	VARCHAR(10)	PK, FK	Không		Tham chiếu PHIEU_XUAT(MaPX)	Phiếu xuất
MaVT	VARCHAR(10)	PK, FK	Không		Tham chiếu VAT_TU(MaVT)	Vật tư được xuất
SoLuong	DECIMAL(18,2)		Không		CHECK > 0	Số lượng xuất

Bảng 2.12. Mô tả bảng PHIEU_CHUYEN_KHO

Tên cột	Kiểu dữ liệu	Khóa	NULL	Mặc định	Ràng buộc	Ý nghĩa
MaPCK	VARCHAR(10)	PK	Không		Duy nhất	Mã phiếu chuyển kho
MaKhoNgon	VARCHAR(10)	FK	Không		Tham chiếu KHO(MaKho)	Mã kho chuyển đi
MaKhoDich	VARCHAR(10)	FK	Không		Tham chiếu KHO(MaKho); khác MaKhoNgon	Mã kho chuyển đến

NgàyChuyen	DATETIME		Không	GETDATE ()	DEFAULT	Ngày chuyển kho
NguoiLap	VARCHAR(10)	FK	Không		Tham chiếu TAI_KHOAN (MaTK)	Người lập phiếu
GhiChu	NVARCHAR(500)		Có			Ghi chú của phiếu
TrangThai	VARCHAR(20)		Không	NHAP	CHECK IN (NHAP, HOAN_THANH, DA_HUY)	Trạng thái phiếu chuyển

Bảng 2.13. Mô tả bảng CT_CHUYEN_KHO

Tên cột	Kiểu dữ liệu	Khóa	NUL L	Mặ c địn h	Ràng buộc	Ý nghĩa
MaPCK	VARCHAR(10)	PK, FK	Không		Tham chiếu PHIEU_CHUYEN_KHO(MaPCK)	Mã phiếu chuyển kho
MaVT	VARCHAR(10)	PK, FK	Không		Tham chiếu VAT_TU(MaVT)	Mã vật tư chuyển
SoLuong	DECIMAL(18, 2)		Không		CHECK > 0	Số lượng chuyển

Bảng 2.14. Mô tả bảng PHIEU_DIEU_CHINH

Tên cột	Kiểu dữ liệu	Khóa	NULL	Mặc định	Ràng buộc	Ý nghĩa
MaPDC	VARCHAR(10)	PK	Không		Duy nhất	Mã phiếu điều chỉnh
MaKho	VARCHAR(10)	FK	Không		Tham chiếu KHO(MaKho)	Mã kho điều chỉnh

NgàyDieu Chinh	DATETIME		Không	GETDATE ()	DEFAULT	Ngày điều chỉnh
NguoiLap	VARCHAR(10)	FK	Không		Tham chiếu TAI_KHOAN (MaTK)	Người lập phiếu
LyDo	NVARCHAR(500)		Không		CHECK không được để trống	Lý do điều chỉnh
TrangThai	VARCHAR(20)		Không	NHAP	CHECK IN (NHAP, HOAN_THANH, DA_HUY)	Trạng thái phiếu

Bảng 2.15. Mô tả bảng CT_DIEU_CHINH

Tên cột	Kiểu dữ liệu	Khóa	NUL L	Mặ c địn h	Ràng buộc	Ý nghĩ a
MaPDC	VARCHAR(10)	PK, FK	Khôn g		Tham chiếu PHIEU_DIEU_CHIN H (MaPDC)	Mã phiếu điều chỉnh
MaVT	VARCHAR(10)	PK, FK	Khôn g		Tham chiếu VAT_TU(MaVT)	Mã vật tư
SoLuongTruo c	DECIMAL(18,2)		Khôn g		CHECK > 0	Số lượn g trước điều chỉnh
SoLuongSau	DECIMAL(18,2)		Khôn g		CHECK ≥ 0 và khác SoLuongTruoc	Số lượn g sau

						điều chỉnh
--	--	--	--	--	--	---------------

CHƯƠNG 3: XÂY DỰNG CƠ SỞ DỮ LIỆU

3.1. Triển khai cơ sở dữ liệu trên SQL Server

Dựa trên mô hình quan hệ và các ràng buộc dữ liệu đã được thiết kế tại Chương 2, nhóm tiến hành hiện thực hóa cơ sở dữ liệu QuanLyVatTuXayDung trên hệ quản trị Microsoft SQL Server.

Quá trình triển khai bao gồm việc khởi tạo cơ sở dữ liệu và thực thi mã nguồn DDL để thiết lập cấu trúc 15 bảng cùng hệ thống khóa chính, khóa ngoại, ràng buộc miền giá trị (CHECK, DEFAULT, UNIQUE). Đoạn mã khởi tạo cơ sở dữ liệu trên hệ thống:

```
USE master;

GO

IF DB_ID(N'QuanLyVatTuXayDung') IS NULL

    CREATE DATABASE QuanLyVatTuXayDung;

GO

USE QuanLyVatTuXayDung;
```

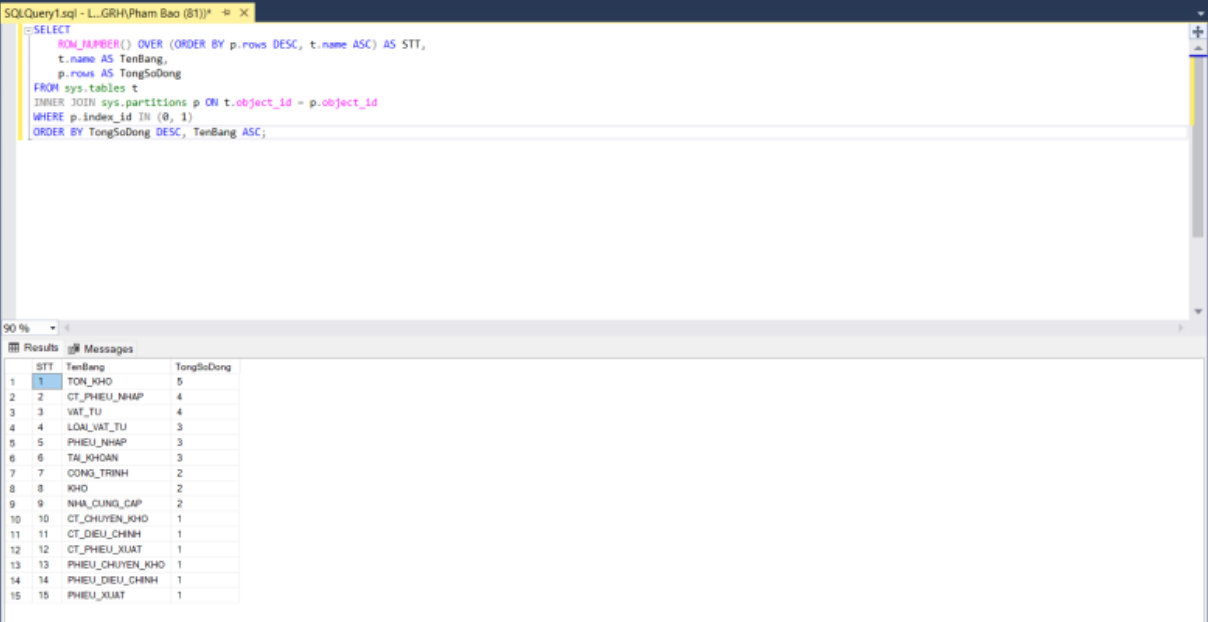
GO

3.2. Khởi tạo dữ liệu mẫu

Sau khi tạo lập cấu trúc cơ sở dữ liệu, nhóm tiến hành nạp bộ dữ liệu mẫu mô phỏng quy trình hoạt động thực tế của doanh nghiệp để phục vụ kiểm thử ràng buộc, xây dựng đối tượng nghiệp vụ và thực nghiệm giao tác.

1. Kiểm tra số lượng bản ghi khởi tạo trên các bảng

Để đảm bảo toàn bộ 15 bảng đều được nạp dữ liệu đầy đủ và không phát sinh lỗi khóa ngoại trong quá trình thực thi, nhóm sử dụng truy vấn đếm số dòng thực tế từ bảng hệ thống:



```

SELECT
    ROW_NUMBER() OVER (ORDER BY p.rows DESC, t.name ASC) AS STT,
    t.name AS TenBang,
    p.rows AS TongSoDong
FROM sys.tables t
INNER JOIN sys.partitions p ON t.object_id = p.object_id
WHERE p.index_id IN (0, 1)
ORDER BY TongSoDong DESC, TenBang ASC;

```

STT	TenBang	TongSoDong
1	TON_KHO	5
2	CT_PHEU_NHAP	4
3	VAT_TU	4
4	LON_VAT_TU	3
5	PHEU_NHAP	3
6	TAL_KHOAN	3
7	CONG_TRINH	2
8	KHO	2
9	NHA_CUNG_CAP	2
10	CT_CHUYEN_KHO	1
11	CT_DIEU_CHINH	1
12	CT_PHEU_XUAT	1
13	PHEU_CHUYEN_KHO	1
14	PHEU_DIEU_CHINH	1
15	PHEU_XUAT	1

2. Trạng thái số dư tồn kho phục vụ thực nghiệm

Dữ liệu tồn kho được hình thành sau chuỗi giao dịch mẫu ban đầu (nhập hàng, xuất công trình, điều chuyển kho và kiểm kê điều chỉnh):

SQLQuery1.sql - L...GRH\Pham Bao (81)P*

```
SELECT * FROM TON_KHO;
```

Results Messages

	MaKho	MaVT	SoLuongTon
1	KHO01	VT01	465.00
2	KHO01	VT02	200.00
3	KHO02	VT01	10.00
4	KHO02	VT03	1000.00
5	KHO02	VT04	800.00

Query executed successfully.

LAPTOP-M3UJ7GRH\SQLEXPRESS ... LAPTOP-M3UJ7GRH\Pham B... QueryVatTuXayDung 00:00:00 5 rows

Ghi chú dữ liệu thực nghiệm: Mặt hàng VT01 (Thép phi 10) tại kho KHO01 ghi nhận số lượng tồn chuẩn là 465.00 cây (kết quả sau chuỗi nghiệp vụ: Nhập 500 Xuất 20 Chuyển 10 Kiểm kê giảm 5). Giá trị này được sử dụng làm trạng thái gốc để phục vụ các kịch bản kiểm thử giao tác và kiểm soát tương tranh .

3.3. Thiết kế và cài đặt các nghiệp vụ chính của hệ thống

3.3.1. Xây dựng Stored Procedure

Code Stored Procedure **sp_NhapKho**

Giải thích Stored Procedure **sp_NhapKho**

a. Mục đích

Thủ tục **sp_NhapKho** được xây dựng để xử lý nghiệp vụ nhập vật tư vào kho. Thủ tục có nhiệm vụ tiếp nhận thông tin nhập kho, kiểm tra tính hợp lệ của dữ liệu trước khi ghi nhận, đồng thời đảm bảo quá trình cập nhật dữ liệu được thực hiện an toàn và chính xác.

b. Tham số đầu vào

Thủ tục **sp_NhapKho** nhận các thông tin dữ liệu cần thiết cho nghiệp vụ nhập kho gồm mã phiếu nhập, mã nhà cung cấp, mã kho, ngày nhập, người lập, ghi chú, trạng thái, mã vật tư, số lượng và đơn giá.

Các tham số này được truyền vào khi gọi thủ tục, giúp hệ thống xác định được nguồn hàng nhập, vị trí lưu kho và thông tin chi tiết của vật tư cần cập nhật.

c. Giải thích logic xử lý

Thủ tục sp_NhapKho được chia thành các phần xử lý chính như sau:

1. Kiểm tra dữ liệu đầu vào

Trước khi thực hiện các thao tác trên cơ sở dữ liệu, thủ tục tiến hành kiểm tra tính hợp lệ của dữ liệu nhận vào.

Cụ thể, hệ thống kiểm tra số lượng nhập phải lớn hơn 0. Đồng thời thực hiện đối chiếu sự tồn tại của các dữ liệu liên quan gồm mã nhà cung cấp trong bảng NHA_CUNG_CAP, mã kho trong bảng KHO và mã vật tư trong bảng VAT_TU. Nếu phát hiện dữ liệu không hợp lệ, hệ thống sử dụng RAISERROR để thông báo lỗi và kết thúc xử lý bằng RETURN, tránh việc ghi nhận dữ liệu không chính xác vào hệ thống.

2. Kiểm soát quá trình thực thi bằng TRY...CATCH

Toàn bộ quá trình xử lý nhập kho được đặt trong cấu trúc TRY...CATCH nhằm quản lý các lỗi có thể xảy ra trong quá trình thực hiện.

Khi có lỗi phát sinh trong quá trình thực thi như lỗi do vi phạm ràng buộc dữ liệu, trùng khóa chính hoặc lỗi trong quá trình thao tác với cơ sở dữ liệu, chương trình sẽ chuyển sang khối CATCH để xử lý.

3. Thực hiện ghi nhận dữ liệu nhập kho

Sau khi dữ liệu được xác nhận hợp lệ, thủ tục tiến hành mở giao tác bằng BEGIN TRANSACTION để quản lý toàn bộ quá trình thay đổi dữ liệu.

Các thao tác xử lý nghiệp vụ nhập kho được thực hiện trong cùng một giao tác, giúp đảm bảo dữ liệu giữa các bảng liên quan luôn ở trạng thái đồng nhất. Chi tiết trình tự thực hiện của giao tác được trình bày tại mục 3.4.2. Phân tích Transaction Nhập kho.

4. Xử lý kết thúc giao tác

Nếu quá trình xử lý không phát sinh lỗi, thủ tục thực hiện COMMIT TRANSACTION để xác nhận và lưu toàn bộ thay đổi vào cơ sở dữ liệu, đồng thời hiển thị thông báo "Nhập kho thành công!".

Trong trường hợp xảy ra lỗi, hệ thống kiểm tra trạng thái giao tác thông qua biến @@TRANCOUNT. Nếu vẫn còn giao tác đang mở, lệnh ROLLBACK TRANSACTION được thực hiện để hủy toàn bộ thay đổi chưa hoàn tất.

Sau đó, lỗi được lấy thông qua hàm ERROR_MESSAGE() và trả về cho người dùng để xác định nguyên nhân xảy ra lỗi.

Code Stored Procedure sp_Xuatkho

Giải thích Stored Procedure sp_Xuatkho

Thủ tục sp_XuatKho thực hiện nghiệp vụ xuất vật tư ra khỏi kho: kiểm tra tính hợp lệ của dữ liệu đầu vào, kiểm tra tồn kho, sau đó ghi nhận phiếu xuất và cập nhật lại số lượng tồn.

Tham số đầu vào của thủ tục gồm @MaPX, @MaKho, @MaCT, @NgayXuat, @NguoiLap, @GhiChu, @TrangThai dùng để xác định thông tin của phiếu xuất. Trong đó, @MaVT và @SoLuong xác định vật tư cần xuất và số lượng thực hiện xuất kho.

Thủ tục đầu tiên kiểm tra điều kiện số lượng xuất. Nếu @SoLuong ≤ 0 , hệ thống sẽ báo lỗi và dừng xử lý bằng lệnh RETURN, đảm bảo không xảy ra trường hợp xuất vật tư với số lượng không hợp lệ.

Tiếp theo, hệ thống kiểm tra sự tồn tại của mã kho và mã vật tư thông qua các bảng KHO và VAT_TU. Việc kiểm tra này giúp đảm bảo phiếu xuất chỉ được tạo với những kho và vật tư đã tồn tại trong hệ thống.

Sau đó, thủ tục kiểm tra số lượng tồn trong bảng TON_KHO dựa trên cặp giá trị (MaKho, MaVT). Nếu không tìm thấy dữ liệu tồn kho hoặc số lượng tồn nhỏ hơn số lượng cần xuất, hệ thống sẽ báo lỗi và không cho phép thực hiện nghiệp vụ, nhằm đảm bảo không xuất vượt quá số lượng vật tư thực tế.

Khi các điều kiện kiểm tra đều hợp lệ, thủ tục thực hiện giao tác bằng BEGIN TRANSACTION. Hệ thống lần lượt thêm dữ liệu vào bảng PHIEU_XUAT, thêm chi tiết vật tư vào bảng CT_PHIEU_XUAT và cập nhật giảm số lượng tồn trong bảng TON_KHO. Các thao tác này được thực hiện trong cùng một giao tác để đảm bảo tính toàn vẹn dữ liệu, tất cả thao tác phải cùng thành công hoặc toàn bộ thay đổi sẽ được hủy bỏ.

Trong quá trình thực hiện, khối TRY...CATCH được sử dụng để xử lý lỗi phát sinh. Nếu xảy ra lỗi như vi phạm ràng buộc hoặc lỗi hệ thống, khối CATCH sẽ kiểm tra @@TRANCOUNT > 0 và thực hiện ROLLBACK TRANSACTION để hoàn tác dữ liệu, sau đó trả thông báo lỗi thông qua RAISERROR.

Code Stored Procedure sp_ChuyenKho

```
CREATE OR ALTER PROCEDURE sp_ChuyenKho
```

```
    @MaPCK VARCHAR(10),
```

```
    @MaKhoNgon VARCHAR(10),
```

```
    @MaKhoDich VARCHAR(10),
```

```
    @MaVT VARCHAR(10),
```

```
    @SoLuongChuyen DECIMAL(18,2),
```

```
    @NguoiLap VARCHAR(10)
```

```
AS
```

```
BEGIN
```

```
    SET NOCOUNT ON;
```

```
    BEGIN TRY
```

```
        -- Kiểm tra điều kiện số lượng
```

```
        IF @SoLuongChuyen <= 0
```

```
        BEGIN
```

```
            RAISERROR(N'Lỗi: Số lượng chuyển phải lớn hơn 0!', 16, 1);
```

```
            RETURN;
```

```
        END
```

```
        -- Kiểm tra kho nguồn và kho đích
```

```
        IF @MaKhoNgon = @MaKhoDich
```

```
        BEGIN
```

```
RAISERROR(N'Lỗi: Kho nguồn và kho đích không được trùng nhau!', 16, 1);  
RETURN;  
END
```

```
-- Kiểm tra kho tồn tại
```

```
IF NOT EXISTS (SELECT 1 FROM KHO WHERE MaKho = @MaKhoNguon)  
BEGIN  
    RAISERROR(N'Lỗi: Kho nguồn không tồn tại!', 16, 1);  
    RETURN;  
END
```

```
IF NOT EXISTS (SELECT 1 FROM KHO WHERE MaKho = @MaKhoDich)  
BEGIN  
    RAISERROR(N'Lỗi: Kho đích không tồn tại!', 16, 1);  
    RETURN;  
END
```

```
-- Kiểm tra vật tư tồn tại
```

```
IF NOT EXISTS (SELECT 1 FROM VAT_TU WHERE MaVT = @MaVT)  
BEGIN  
    RAISERROR(N'Lỗi: Vật tư không tồn tại!', 16, 1);  
    RETURN;  
END
```

```
-- Kiểm tra mã phiếu chuyển đã tồn tại
```

```
IF EXISTS (SELECT 1 FROM PHIEU_CHUYEN_KHO WHERE MaPCK =  
@MaPCK)  
BEGIN
```

```

    RAISERROR(N'Lỗi: Mã phiếu chuyển kho đã tồn tại!', 16, 1);
    RETURN;
END

```

```

DECLARE @TonKhoNgon DECIMAL(18,2);

```

```

-- Bắt đầu giao tác

```

```

BEGIN TRANSACTION;

```

```

-- Kiểm tra tồn kho nguồn

```

```

SELECT @TonKhoNgon = SoLuongTon

```

```

FROM TON_KHO

```

```

WHERE MaKho = @MaKhoNgon AND MaVT = @MaVT;

```

```

IF @TonKhoNgon IS NULL OR @TonKhoNgon < @SoLuongChuyen

```

```

BEGIN

```

```

    RAISERROR(N'Lỗi: Số lượng tồn ở kho nguồn không đủ hoặc không tồn tại!',
16, 1);

```

```

    ROLLBACK TRANSACTION;

```

```

    RETURN;

```

```

END

```

```

-- 1. Insert phiếu chuyển kho

```

```

INSERT INTO PHIEU_CHUYEN_KHO (MaPCK, MaKhoNgon, MaKhoDich,
NgayChuyen, NguoiLap, TrangThai)

```

```

VALUES (@MaPCK, @MaKhoNgon, @MaKhoDich, GETDATE(),
@NguoiLap, N'HOAN_THANH');

```

-- 2. Insert chi tiết chuyển kho

```
INSERT INTO CT_CHUYEN_KHO (MaPCK, MaVT, SoLuong)
VALUES (@MaPCK, @MaVT, @SoLuongChuyen);
```

-- 3. Trừ tồn kho nguồn

```
UPDATE TON_KHO
SET SoLuongTon = SoLuongTon - @SoLuongChuyen
WHERE MaKho = @MaKhoNgon AND MaVT = @MaVT;
```

-- 4. Cộng tồn kho đích

```
IF EXISTS (SELECT 1 FROM TON_KHO WHERE MaKho = @MaKhoDich
AND MaVT = @MaVT)
```

```
BEGIN
```

```
    UPDATE TON_KHO
```

```
    SET SoLuongTon = SoLuongTon + @SoLuongChuyen
```

```
    WHERE MaKho = @MaKhoDich AND MaVT = @MaVT;
```

```
END
```

```
ELSE
```

```
BEGIN
```

```
    INSERT INTO TON_KHO (MaKho, MaVT, SoLuongTon)
```

```
    VALUES (@MaKhoDich, @MaVT, @SoLuongChuyen);
```

```
END
```

-- Commit giao tác

```
COMMIT TRANSACTION;
```

```
PRINT N'Chuyển kho thành công!';
```

```
END TRY
```

```

BEGIN CATCH
    IF @@TRANCOUNT > 0
        ROLLBACK TRANSACTION;

    DECLARE @ErrorMessage NVARCHAR(4000);
    SET @ErrorMessage = ERROR_MESSAGE();

    RAISERROR(@ErrorMessage, 16, 1);
END CATCH
END;

```

Giải thích Stored Procedure sp_ChuyenKho

a. Mục đích

Thủ tục **sp_ChuyenKho** được xây dựng để xử lý nghiệp vụ chuyển vật tư giữa hai kho trong hệ thống. Thủ tục có nhiệm vụ tiếp nhận thông tin chuyển kho, kiểm tra tính hợp lệ của dữ liệu trước khi ghi nhận, đồng thời đảm bảo quá trình cập nhật tồn kho tại kho nguồn và kho đích được thực hiện an toàn, đồng bộ và chính xác.

b. Tham số đầu vào

Thủ tục **sp_ChuyenKho** nhận các thông tin dữ liệu cần thiết cho nghiệp vụ chuyển kho gồm mã phiếu chuyển kho, mã kho nguồn, mã kho đích, mã vật tư, số lượng chuyển và người lập.

Các tham số này được truyền vào khi gọi thủ tục, giúp hệ thống xác định được kho xuất, kho nhận và thông tin chi tiết của vật tư cần cập nhật.

c. Giải thích logic xử lý

Thủ tục sp_ChuyenKho được chia thành các phần xử lý chính như sau:

1. Kiểm tra dữ liệu đầu vào

Trước khi thực hiện các thao tác trên cơ sở dữ liệu, thủ tục tiến hành kiểm tra tính hợp lệ của dữ liệu nhận vào. Cụ thể, hệ thống kiểm tra số lượng chuyển phải lớn hơn 0 và kho nguồn phải khác kho đích. Đồng thời thực hiện đối chiếu sự tồn tại của các dữ liệu liên quan gồm mã kho nguồn, mã kho đích trong bảng KHO, mã vật tư trong bảng VAT_TU, và kiểm tra mã phiếu chuyển kho chưa từng tồn tại trong bảng PHIEU_CHUYEN_KHO để tránh trùng khóa chính. Nếu phát hiện dữ liệu không hợp lệ, hệ thống sử dụng RAISERROR để thông báo lỗi và kết thúc xử lý bằng RETURN, tránh việc ghi nhận dữ liệu không chính xác vào hệ thống.

2. Kiểm soát quá trình thực thi bằng TRY...CATCH

Toàn bộ quá trình xử lý chuyển kho được đặt trong cấu trúc TRY...CATCH nhằm quản lý các lỗi có thể xảy ra trong quá trình thực hiện. Khi có lỗi phát sinh trong quá trình thực thi như lỗi do vi phạm ràng buộc dữ liệu, trùng khóa chính hoặc lỗi trong quá trình thao tác với cơ sở dữ liệu, chương trình sẽ chuyển sang khối CATCH để xử lý.

3. Kiểm tra tồn kho nguồn và thực hiện ghi nhận dữ liệu chuyển kho

Sau khi dữ liệu đầu vào được xác nhận hợp lệ, thủ tục mở giao tác bằng BEGIN TRANSACTION để quản lý toàn bộ quá trình thay đổi dữ liệu. Trong giao tác, hệ thống truy vấn số lượng tồn hiện tại của vật tư tại kho nguồn; nếu không tồn tại dòng tồn kho tương ứng hoặc số lượng tồn nhỏ hơn số lượng cần chuyển, giao tác được hủy bằng ROLLBACK TRANSACTION và thủ tục dừng lại. Các thao tác xử lý nghiệp vụ chuyển kho được thực hiện trong cùng một giao tác, giúp đảm bảo dữ liệu giữa các bảng liên quan luôn ở trạng thái đồng nhất. Chi tiết trình tự thực hiện của giao tác được trình bày tại mục 3.4.2 - Transaction nghiệp vụ chuyển kho.

4. Xử lý kết thúc giao tác

Nếu quá trình xử lý không phát sinh lỗi, thủ tục thực hiện COMMIT TRANSACTION để xác nhận và lưu toàn bộ thay đổi vào cơ sở dữ liệu, đồng thời hiển thị thông báo "Chuyển kho thành công!". Trong trường hợp xảy ra lỗi, hệ thống kiểm tra trạng thái giao tác thông qua biến @@TRANCOUNT; nếu vẫn còn giao tác đang mở, lệnh ROLLBACK TRANSACTION được thực hiện để hủy toàn bộ thay đổi chưa hoàn tất.

Sau đó, lỗi được lấy thông qua hàm `ERROR_MESSAGE()` và trả về cho người dùng thông qua `RAISERROR` để xác định nguyên nhân xảy ra lỗi.

3.3.2. Xây dựng View

3.3.3. Xây dựng Trigger

Để tránh việc nhân viên xóa nhầm các mặt hàng vẫn còn hàng tồn thực tế trong kho, nhóm cài đặt trigger `trg_VatTu_NgauXoa` trên bảng `VAT_TU`

Trigger sử dụng cơ chế `INSTEAD OF DELETE`: khi nhận thao tác xóa vật tư, hệ thống sẽ kiểm tra số lượng tồn trong bảng `TON_KHO`. Nếu mặt hàng đó vẫn còn số lượng tồn > 0 tại bất kỳ kho nào, hệ thống lập tức hủy giao tác (rollback) và gửi cảnh báo lỗi. Nếu số lượng tồn đã bằng 0, hệ thống tự động cập nhật `TrangThai = 0` (chuyển sang trạng thái ngưng hoạt động) nhằm bảo toàn dữ liệu lịch sử trên các hóa đơn và chứng từ cũ.

```
USE QuanLyVatTuXayDung;

GO

-- Trigger chặn xóa vật tư nếu vẫn còn tồn kho

CREATE OR ALTER TRIGGER trg_VatTu_NgauXoa

ON VAT_TU

INSTEAD OF DELETE

AS

BEGIN

    SET NOCOUNT ON;

    -- Kiểm tra nếu vật tư yêu cầu xóa vẫn còn số lượng tồn > 0 tại bất kỳ kho nào

    IF EXISTS (

        SELECT 1

        FROM TON_KHO tk
```

```

        JOIN deleted d ON tk.MaVT = d.MaVT

        WHERE tk.SoLuongTon > 0

    )

    BEGIN

        RAISERROR (N'Lỗi nghiệp vụ: Không thể xóa vật tư vì vẫn còn số
        lượng tồn trong kho!', 16, 1);

        ROLLBACK TRANSACTION;

        RETURN;

    END;

    -- Nếu tồn kho bằng 0 thì cho phép xóa mềm (chuyển trạng thái sang
    ngưng hoạt động)

    UPDATE VAT_TU

    SET TrangThai = 0

    WHERE MaVT IN (SELECT MaVT FROM deleted);

    PRINT N'Đã cập nhật trạng thái vật tư sang ngưng hoạt động thành
    công.';

    END;

    GO

```

Kiểm thử kích hoạt trigger: Thực hiện xóa thử mặt hàng thép phi 10 (VT01) đang còn tồn kho 465 cây tại KHO01:

```
DELETE FROM VAT_TU WHERE MaVT = 'VT01';
```

3.4. Phân tích và xây dựng giao tác cho các nghiệp vụ cần đảm bảo toàn vẹn

3.4.1. Tổng quan cơ chế khóa và cấp độ cô lập trong SQL Server

- Cơ chế khóa trong SQL Server
- Shared Lock
- Exclusive Lock

Isolation Level:

- Read Uncommitted
- Read Committed
- Repeatable Read
- Serializable

3.4.2. Xây dựng Transaction cho các nghiệp vụ chính**Transaction nghiệp vụ nhập kho**

Transaction nhập kho bao gồm các thao tác thêm phiếu nhập, thêm chi tiết phiếu nhập và cập nhật tồn kho. Giao tác được sử dụng nhằm đảm bảo các thao tác này được thực hiện đầy đủ và đồng bộ. Nếu xảy ra lỗi trong quá trình thực hiện, toàn bộ thay đổi sẽ bị hủy bỏ.

Bước 1: Insert phiếu nhập:

Hệ thống tạo bản ghi mới trong bảng PHIEU_NHAP để lưu thông tin chung của lần nhập kho như mã phiếu nhập, nhà cung cấp, kho nhập, ngày nhập và người lập.

Bước 2: Insert chi tiết phiếu nhập:

Hệ thống thêm thông tin vật tư nhập vào bảng CT_PHIEU_NHAP, bao gồm mã vật tư, số lượng và đơn giá. Dữ liệu này được liên kết với phiếu nhập thông qua mã phiếu nhập.

Bước 3: Cập nhật cộng tồn kho:

Hệ thống cập nhật bảng TON_KHO để phản ánh số lượng vật tư sau khi nhập. Nếu vật tư đã tồn tại trong kho, số lượng tồn được cộng thêm số lượng nhập mới. Nếu chưa có, hệ thống tạo mới bản ghi tồn kho.

Bước 4: Commit hoặc Rollback:

Commit: Khi các bước trên thực hiện thành công, lệnh COMMIT TRANSACTION được sử dụng để lưu toàn bộ thay đổi vào cơ sở dữ liệu.

Rollback: Nếu xảy ra lỗi trong quá trình thực hiện, lệnh ROLLBACK TRANSACTION được gọi để hủy toàn bộ thay đổi, đảm bảo dữ liệu không bị sai lệch.

Transaction nghiệp vụ xuất kho

Nghiệp vụ xuất kho liên quan đến nhiều bảng dữ liệu có quan hệ ràng buộc với nhau gồm PHIEU_XUAT, CT_PHIEU_XUAT và TON_KHO. Vì vậy, quá trình xử lý cần được thực hiện bằng transaction nhằm đảm bảo tính toàn vẹn dữ liệu theo nguyên tắc **all-or-nothing** (tất cả thành công hoặc không có thay đổi nào được ghi nhận).

Quy trình xử lý transaction trong nghiệp vụ xuất kho gồm các bước sau:

Bước 1: Kiểm tra tồn kho

Trước khi bắt đầu transaction, hệ thống kiểm tra các điều kiện đầu vào gồm số lượng xuất phải lớn hơn 0, mã kho và mã vật tư phải tồn tại trong hệ thống. Đồng thời, hệ thống kiểm tra SoLuongTon trong bảng TON_KHO có đủ để thực hiện xuất kho hay không. Nếu điều kiện không hợp lệ, thủ tục sẽ dừng xử lý và không thực hiện giao tác.

Bước 2: Tạo phiếu xuất

Sau khi kiểm tra hợp lệ, hệ thống bắt đầu transaction bằng BEGIN TRANSACTION. Thủ tục thực hiện thêm thông tin phiếu xuất vào bảng PHIEU_XUAT và thêm chi tiết vật tư xuất vào bảng CT_PHIEU_XUAT. Hai thao tác này được thực hiện trong cùng một giao tác nhằm đảm bảo thông tin phiếu xuất và chi tiết vật tư luôn đồng bộ.

Bước 3: Cập nhật tồn kho

Hệ thống thực hiện cập nhật bảng TON_KHO bằng cách giảm SoLuongTon tương ứng với số lượng vật tư đã xuất. Đây là thao tác ghi dữ liệu nên SQL Server sẽ áp dụng cơ chế khóa phù hợp để tránh các giao tác khác đồng thời thay đổi cùng một dữ liệu gây sai lệch số lượng tồn kho.

Bước 4: Commit hoặc Rollback

Nếu các thao tác tạo phiếu xuất, tạo chi tiết phiếu xuất và cập nhật tồn kho thực hiện thành công, hệ thống sử dụng COMMIT TRANSACTION để ghi nhận thay đổi vào cơ sở dữ liệu. Nếu xảy ra lỗi trong quá trình thực hiện, khối CATCH sẽ kiểm tra giao tác đang hoạt động và thực hiện ROLLBACK TRANSACTION để hủy toàn bộ thay đổi.

Cơ chế kết hợp giữa COMMIT và ROLLBACK giúp đảm bảo tính nguyên tử (**Atomicity**) của transaction, tránh xảy ra tình trạng dữ liệu không đồng nhất như đã giảm tồn kho nhưng chưa tạo phiếu xuất hoặc đã tạo phiếu xuất nhưng chưa cập nhật lại tồn kho.

Transaction nghiệp vụ chuyển kho

Transaction chuyển kho bao gồm các thao tác thêm phiếu chuyển kho, thêm chi tiết chuyển kho, trừ tồn kho tại kho nguồn và cộng tồn kho tại kho đích. Vì các thao tác này tác động đến nhiều bảng và nhiều dòng dữ liệu tồn kho khác nhau, giao tác được sử dụng nhằm đảm bảo tất cả các thay đổi được thực hiện đầy đủ và đồng bộ. Nếu xảy ra lỗi ở bất kỳ bước nào trong quá trình thực hiện, toàn bộ thay đổi sẽ bị hủy bỏ để tránh tình trạng dữ liệu tồn kho bị chênh lệch giữa hai kho.

Bước 1: Kiểm tra kho nguồn và kho đích:

Trước khi ghi nhận phiếu, hệ thống kiểm tra kho nguồn và kho đích không được trùng nhau, đồng thời truy vấn số lượng tồn hiện tại của vật tư tại kho nguồn trong bảng TON_KHO. Nếu không tồn tại dòng tồn kho tương ứng hoặc số lượng tồn nhỏ hơn số lượng cần chuyển, giao tác bị hủy ngay để đảm bảo không phát sinh tồn kho âm.

Bước 2: Trừ kho nguồn:

Sau khi hệ thống tạo bản ghi mới trong bảng PHIEU_CHUYEN_KHO và thêm dòng chi tiết vào bảng CT_CHUYEN_KHO, hệ thống cập nhật bảng TON_KHO để giảm số lượng tồn của vật tư tại kho nguồn tương ứng với số lượng đã chuyển đi.

Bước 3: Cộng kho đích:

Hệ thống tiếp tục cập nhật bảng TON_KHO tại kho đích. Nếu vật tư đã tồn tại tại kho đích, số lượng tồn được cộng thêm số lượng vừa chuyển đến; nếu vật tư chưa từng xuất hiện tại kho đích, hệ thống tạo mới một bản ghi tồn kho cho kho đích với số lượng bằng số lượng chuyển.

Bước 4: Commit hoặc Rollback:

Commit: Khi các bước trên thực hiện thành công, lệnh COMMIT TRANSACTION được sử dụng để lưu toàn bộ thay đổi vào cơ sở dữ liệu.

Rollback: Nếu xảy ra lỗi trong quá trình thực hiện (ví dụ: tồn kho nguồn không đủ, hoặc lỗi vi phạm ràng buộc dữ liệu), lệnh ROLLBACK TRANSACTION được gọi để hủy toàn bộ thay đổi, đảm bảo dữ liệu tồn kho tại cả hai kho không bị sai lệch - tránh tình trạng đã trừ được kho nguồn nhưng chưa cộng được kho đích.

Transaction nghiệp vụ điều chỉnh tồn kho

- Lưu số lượng trước điều chỉnh
- Cập nhật số lượng sau điều chỉnh
- Ghi nhận lý do

3.4.3. Mô phỏng tranh chấp dữ liệu và Kiểm thử giải pháp điều khiển tương tranh Xử lý Mất dữ liệu cập nhật (Lost Update)

Mô tả kịch bản: Giả lập tình huống hai thủ kho cùng thao tác đồng thời trên mặt hàng Thép phi 10 (VT01) tại KHO01 (số lượng tồn ban đầu là 465.00 cây):

Session 1 (Thủ kho 1 - Xuất kho): Đọc số lượng tồn hiện tại (465.00), hệ thống gặp độ trễ xử lý 8 giây (WAITFOR DELAY '00:00:08'), sau đó thực hiện trừ 15 cây.

Session 2 (Thủ kho 2 - Nhập kho): Trong lúc Session 1 đang trễ, Session 2 đọc số lượng tồn (vẫn là 465.00), cộng thêm 50 cây và thực hiện COMMIT trước.

Hậu quả: Session 1 hoàn tất sau 8 giây, sử dụng giá trị đọc ban đầu (465.00) để trừ 15 thành 450.00 và ghi đè vào cơ sở dữ liệu. Toàn bộ thao tác nhập 50 cây của Session 2 bị mất hoàn toàn.

<pre>USE QuanLyVatTuXayDung; GO BEGIN TRANSACTION; DECLARE @ton DECIMAL(18,2); SELECT @ton = SoLuongTon FROM TON_KHO WHERE MaKho = 'KHO01' AND MaVT = 'VT01'; WAITFOR DELAY '00:00:08'; -- Đợi 8 giây</pre>	<pre>USE QuanLyVatTuXayDung; GO BEGIN TRANSACTION; DECLARE @ton DECIMAL(18,2); SELECT @ton = SoLuongTon FROM TON_KHO WHERE MaKho = 'KHO01' AND MaVT = 'VT01'; UPDATE TON_KHO SET SoLuongTon = @ton + 50 WHERE MaKho = 'KHO01' AND MaVT = 'VT01'; COMMIT TRANSACTION;</pre>
--	--

<pre> UPDATE TON_KHO SET SoLuongTon = @ton - 15 WHERE MaKho = 'KHO01' AND MaVT = 'VT01'; COMMIT TRANSACTION; PRINT N'Session 1 (Xuất 15) xong!'; </pre>	<pre> PRINT N'Session 2 (Nhập 50) xong!'; </pre>
---	--

Kết quả :

MaKho	MaVT	SoLuongTon
KHO01	VT01	450.00

2. Giải pháp khắc phục sử dụng gợi ý khóa WITH (UPDLOCK)

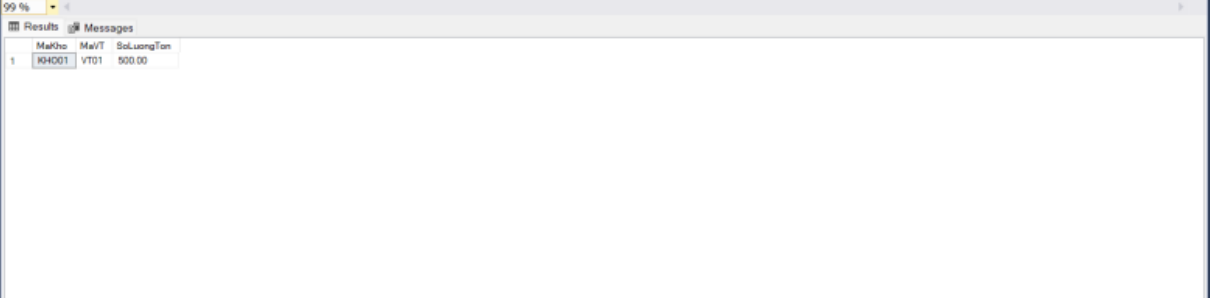
Cơ chế hoạt động: Bổ sung gợi ý khóa WITH (UPDLOCK) vào câu lệnh SELECT đọc số dư tồn kho ở cả hai giao tác. Khi Session 1 đọc dữ liệu với UPDLOCK, SQL Server sẽ cấp quyền khóa cập nhật trên bản ghi của VT01. Session 2 khi cố gắng đọc dữ liệu với UPDLOCK sẽ lập tức bị đưa vào trạng thái chờ (Blocked/Waiting) cho đến khi Session 1 kết thúc (COMMIT). Khi đó, Session 2 mới đọc được số dư mới nhất (450.00) và cộng tiếp 50 thành 500.00.

Mã nguồn xử lý lỗi:

<pre> USE QuanLyVatTuXayDung; GO BEGIN TRANSACTION; DECLARE @ton DECIMAL(18,2); SELECT @ton = SoLuongTon FROM TON_KHO WITH (UPDLOCK) </pre>	<pre> USE QuanLyVatTuXayDung; GO BEGIN TRANSACTION; DECLARE @ton DECIMAL(18,2); SELECT @ton = SoLuongTon FROM TON_KHO WITH (UPDLOCK) </pre>
--	--

<pre> WHERE MaKho = 'KHO01' AND MaVT = 'VT01'; WAITFOR DELAY '00:00:08'; UPDATE TON_KHO SET SoLuongTon = @ton - 15 WHERE MaKho = 'KHO01' AND MaVT = 'VT01'; COMMIT TRANSACTION; PRINT N'Session 1 (UPDLOCK) hoàn tất!'; </pre>	<pre> WHERE MaKho = 'KHO01' AND MaVT = 'VT01'; UPDATE TON_KHO SET SoLuongTon = @ton + 50 WHERE MaKho = 'KHO01' AND MaVT = 'VT01'; COMMIT TRANSACTION; PRINT N'Session 2 (UPDLOCK) hoàn tất!'; SELECT MaKho, MaVT, SoLuongTon FROM TON_KHO WHERE MaKho = 'KHO01' AND MaVT = 'VT01'; </pre>
--	---

Kết quả :



MaKho	MaVT	SoLuongTon
KHO01	VT01	500.00

Xử lý Đọc dữ liệu rác (Dirty Read)

a. Mục đích thực nghiệm

Dirty Read là hiện tượng một giao tác đọc được dữ liệu do giao tác khác thay đổi nhưng chưa được xác nhận bằng lệnh COMMIT TRANSACTION. Nếu giao tác thực hiện thay đổi sau đó bị hủy bằng ROLLBACK TRANSACTION, dữ liệu mà giao tác còn lại đã đọc sẽ trở thành dữ liệu không hợp lệ.

Mục tiêu của thực nghiệm là mô phỏng lỗi Dirty Read trong quá trình truy xuất dữ liệu tồn kho và kiểm tra khả năng khắc phục bằng mức cô lập READ COMMITTED trong SQL Server.

b. Kịch bản thực nghiệm

Thực nghiệm được thực hiện trên bảng TON_KHO với vật tư VT01 tại kho KHO01 bằng hai Session độc lập trong SQL Server Management Studio.

Quá trình kiểm thử gồm hai trường hợp:

Trường hợp 1: Sử dụng mức cô lập READ UNCOMMITTED để mô phỏng lỗi Dirty Read.

Trường hợp 2: Thay đổi sang mức cô lập READ COMMITTED để kiểm tra khả năng ngăn chặn lỗi.

c. Code thực nghiệm

Lưu ý: Thực nghiệm được thực hiện trên hai cửa sổ Query độc lập trong SQL Server Management Studio. Session 1 và Session 2 được chạy đồng thời để mô phỏng tình huống tranh chấp dữ liệu giữa các giao tác.

Session 1: Mô phỏng giao tác cập nhật chưa hoàn tất

```
BEGIN TRANSACTION;

UPDATE TON_KHO
SET SoLuongTon = 999.00
WHERE MaKho = 'KHO01' AND MaVT = 'VT01';

WAITFOR DELAY '00:00:10';

ROLLBACK TRANSACTION;

PRINT N'Session 1 đã Rollback hoàn toàn dữ liệu!';
```

Session 2: Đọc dữ liệu bằng READ UNCOMMITTED

```
SET TRANSACTION ISOLATION LEVEL
READ UNCOMMITTED;

SELECT MaKho, MaVT, SoLuongTon
FROM TON_KHO
```

```
SET TRANSACTION ISOLATION LEVEL
READ COMMITTED;

SELECT MaKho, MaVT, SoLuongTon
FROM TON_KHO
```

WHERE MaKho = 'KHO01' AND MaVT = 'VT01';	WHERE MaKho = 'KHO01' AND MaVT = 'VT01';
--	--

d. Phân tích kết quả thực nghiệm

1. Trước khi xử lý – Phát sinh lỗi Dirty Read

Tại Session 1, hệ thống thực hiện cập nhật tạm thời số lượng tồn của vật tư VT01 tại kho KHO01 thành giá trị 999.00. Tuy nhiên, giao tác này chưa được xác nhận bằng lệnh COMMIT TRANSACTION mà vẫn đang trong trạng thái xử lý do có lệnh WAITFOR DELAY.

Trong khoảng thời gian Session 1 chưa hoàn tất giao tác, Session 2 thực hiện truy vấn với mức cô lập:

READ UNCOMMITTED

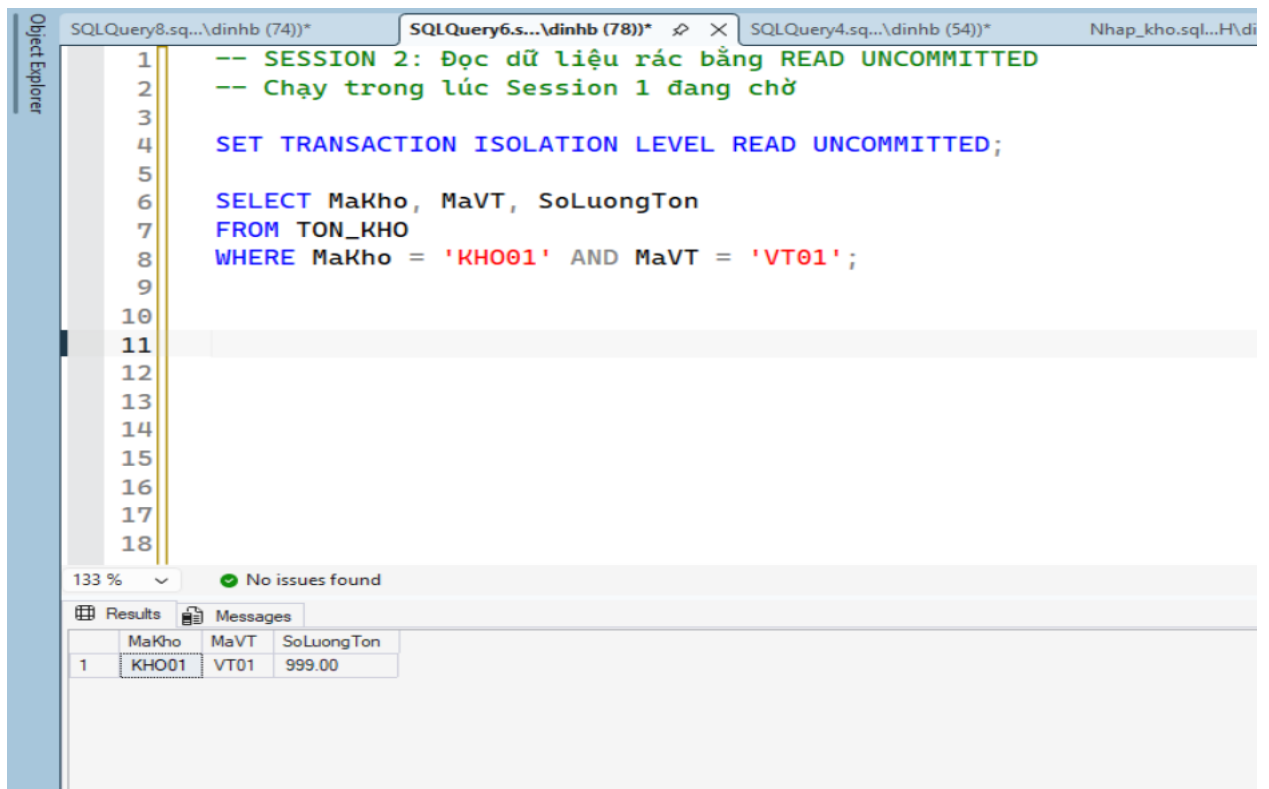
Do mức cô lập này không yêu cầu dữ liệu phải được xác nhận trước khi đọc, Session 2 có thể lấy được giá trị tạm thời 999.00 từ Session 1.

Sau thời gian chờ, Session 1 thực hiện:

ROLLBACK TRANSACTION

làm cho giá trị 999.00 bị hủy bỏ và dữ liệu quay về trạng thái ban đầu.

Như vậy, Session 2 đã đọc được dữ liệu chưa từng tồn tại chính thức trong cơ sở dữ liệu. Đây chính là hiện tượng **Dirty Read**.



Hình 3.4.3.1. Session 2 đọc dữ liệu chưa COMMIT bằng mức cô lập READ UNCOMMITTED

2. Sau khi xử lý – Khắc phục bằng READ COMMITTED

Sau khi thay đổi mức cô lập của Session 2 sang:

READ COMMITTED

SQL Server sẽ không cho phép giao tác đọc dữ liệu đang bị một giao tác khác thay đổi nhưng chưa hoàn tất.

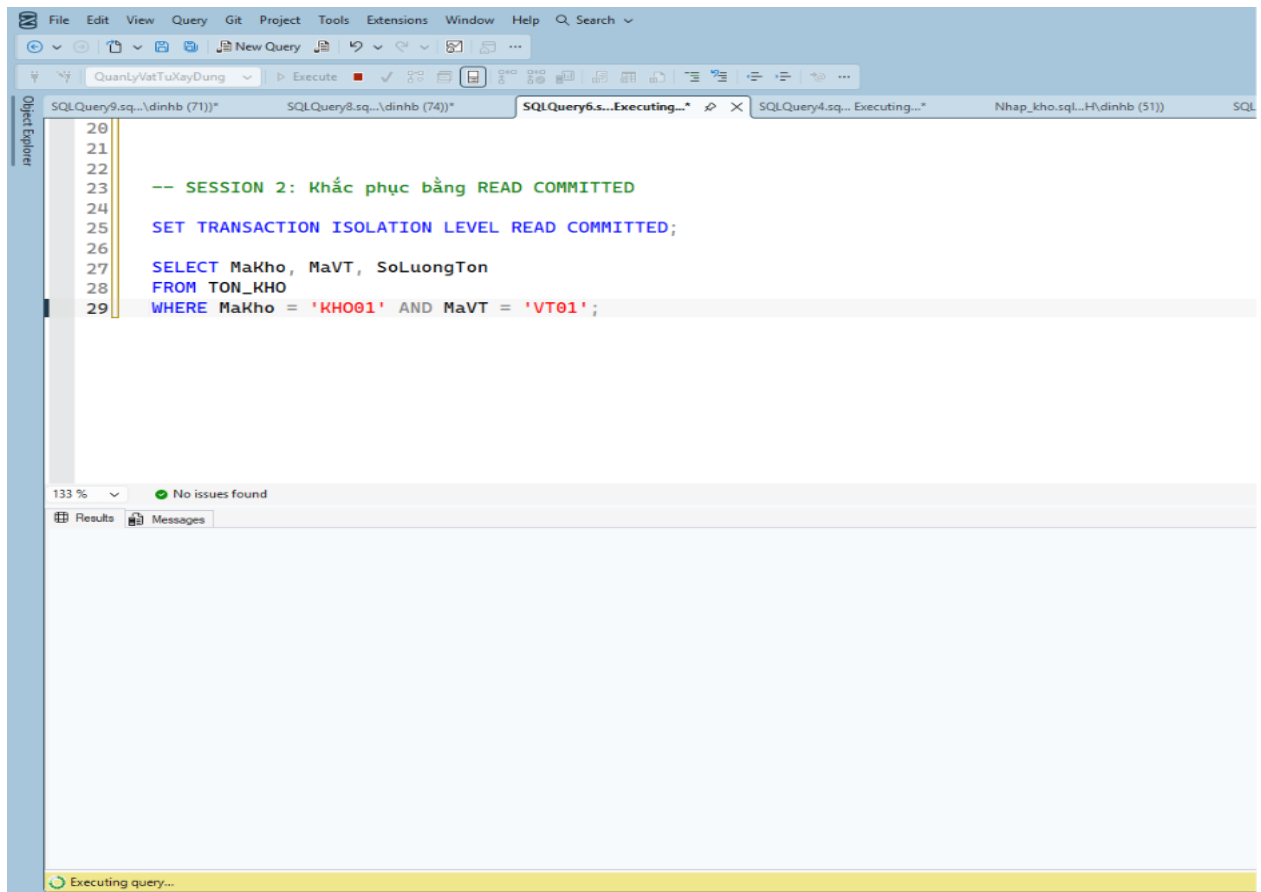
Trong lúc Session 1 vẫn đang giữ giao tác cập nhật, câu lệnh SELECT tại Session 2 sẽ bị tạm dừng và chờ đến khi Session 1 thực hiện COMMIT hoặc ROLLBACK.

Sau khi Session 1 thực hiện:

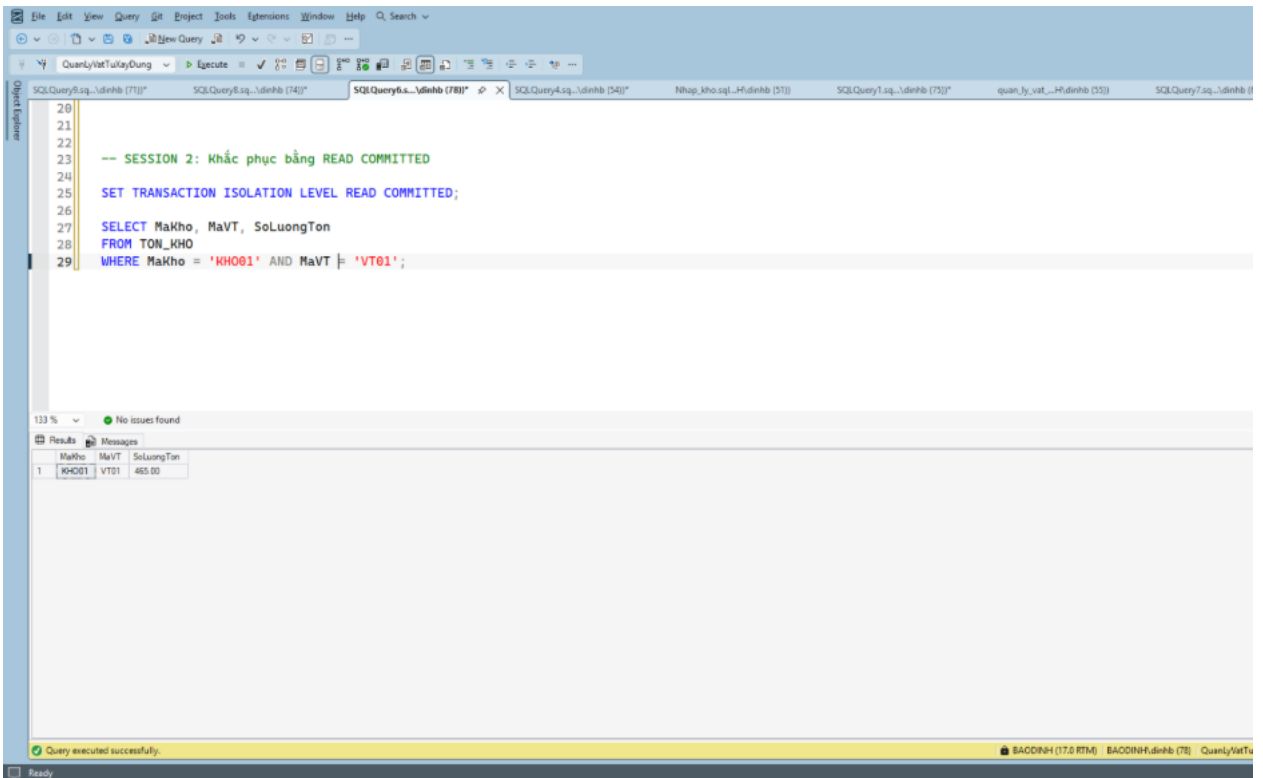
ROLLBACK TRANSACTION

Session 2 tiếp tục thực hiện truy vấn và nhận được giá trị tồn kho ban đầu của vật tư trong cơ sở dữ liệu ($SoLuongTon = 465.00$). Giá trị này là dữ liệu hợp lệ trước khi Session 1 thực hiện thao tác cập nhật tạm thời.

Kết quả cho thấy dữ liệu đọc được là dữ liệu hợp lệ ($SoLuongTon = 465.00$), không còn xuất hiện giá trị tạm thời 999.00 do giao tác trước đó đã bị hủy bằng ROLLBACK TRANSACTION.



Hình 3.4.3.2. Kết quả Session 2 bị chặn khi truy vấn dữ liệu bằng mức cô lập READ COMMITTED



Hình 3.4.3.3. Session 2 trả về dữ liệu tồn kho hợp lệ sau khi Session 1 ROLLBACK

Xử lý Không đọc lại được dữ liệu (Non-repeatable Read)

a. Mục đích thực nghiệm

Non-repeatable Read là hiện tượng xảy ra khi trong cùng một giao tác, hai lần đọc (SELECT) cùng một dữ liệu trả về hai kết quả khác nhau do một giao tác khác đã thực hiện cập nhật (UPDATE) và xác nhận (COMMIT) dữ liệu ở giữa.

Thực nghiệm được thực hiện nhằm kiểm tra hiện tượng thay đổi dữ liệu khi nhiều giao tác cùng truy cập bảng TON_KHO, đồng thời đánh giá khả năng khắc phục bằng mức cô lập REPEATABLE READ để đảm bảo tính nhất quán dữ liệu trong quá trình kiểm tra tồn kho vật tư.

b. Kịch bản thực nghiệm

Thực nghiệm được thực hiện trên bảng TON_KHO của cơ sở dữ liệu QuanLyVatTuXayDung với vật tư VT01 tại kho KHO01, thông qua hai Session độc lập trên SQL Server Management Studio.

Trong trường hợp đầu tiên, Session 1 sử dụng mức cô lập READ COMMITTED, thực hiện đọc số lượng tồn kho lần đầu và tạm dừng giao tác bằng lệnh WAITFOR DELAY. Trong khoảng thời gian này, Session 2 thực hiện cập nhật tăng số lượng tồn kho thêm 100 và COMMIT giao tác.

Sau khi Session 1 tiếp tục thực hiện câu lệnh đọc dữ liệu lần thứ hai, kết quả được so sánh với lần đọc đầu tiên để xác định hiện tượng Non-repeatable Read.

Tiếp theo, thực hiện lại kịch bản trên với mức cô lập REPEATABLE READ. Ở mức này, hệ thống duy trì khóa đọc (Shared Lock) trên dữ liệu đã truy xuất cho đến khi giao tác kết thúc, giúp ngăn các giao tác khác cập nhật dữ liệu đang được sử dụng.

c. Code thực nghiệm

Trường hợp 1: Mô phỏng Non-repeatable Read với READ COMMITTED

```
-- Session 1 SET TRANSACTION ISOLATION LEVEL READ COMMITTED;

BEGIN TRANSACTION;

-- Đọc dữ liệu tồn kho lần thứ nhất

SELECT MaKho, MaVT, SoLuongTon

FROM TON_KHO

WHERE MaKho = 'KHO01' AND MaVT = 'VT01'; --

WAITFOR DELAY '00:00:08';

-- Đọc lại dữ liệu tồn kho

SELECT MaKho, MaVT, SoLuongTon

FROM TON_KHO

WHERE MaKho = 'KHO01'

AND MaVT = 'VT01';

COMMIT TRANSACTION;

PRINT N'Session 1 hoàn tất';
```

```
-- Session 2 -- Thực hiện trong lúc Session 1 đang WAITFOR
BEGIN TRANSACTION;
UPDATE TON_KHO
SET SoLuongTon = SoLuongTon + 100
WHERE MaKho = 'KH001' AND MaVT = 'VT01';
COMMIT TRANSACTION;
PRINT N'Session 2 hoàn tất';
```

Trường hợp 2: Khắc phục bằng REPEATABLE READ

```
-- Session 1: Sử dụng mức cô lập REPEATABLE READ để giữ dữ liệu đọc ổn định
SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;

BEGIN TRANSACTION;

-- Đọc số lượng tồn kho lần đầu
SELECT MaKho, MaVT, SoLuongTon
FROM TON_KHO
WHERE MaKho = 'KH001'
AND MaVT = 'VT01';

-- Tạo khoảng thời gian để Session 2 thực hiện UPDATE
WAITFOR DELAY '00:00:08';

-- Đọc lại dữ liệu để kiểm tra tính nhất quán
SELECT MaKho, MaVT, SoLuongTon
FROM TON_KHO
WHERE MaKho = 'KH001'
AND MaVT = 'VT01';
COMMIT TRANSACTION;

-- Session 2: Thực hiện cập nhật dữ liệu tồn kho
-- Sẽ bị BLOCK nếu Session 1 chưa COMMIT
BEGIN TRANSACTION;

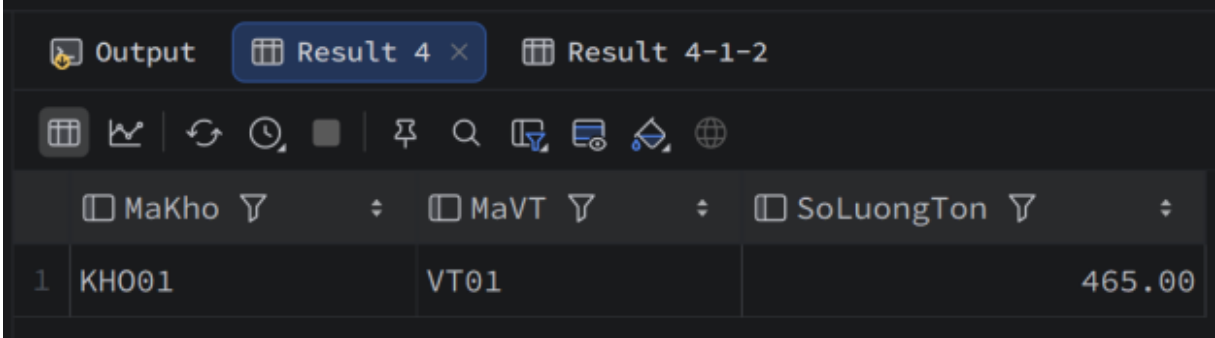
UPDATE TON_KHO
SET SoLuongTon = SoLuongTon + 100
WHERE MaKho = 'KH001'
AND MaVT = 'VT01';

COMMIT TRANSACTION;
```


d. Kết quả thực nghiệm

Kết quả khi sử dụng READ COMMITTED

Khi thực hiện với mức cô lập READ COMMITTED, Session 1 đọc số lượng tồn kho lần đầu nhận được giá trị ban đầu là 465.00.

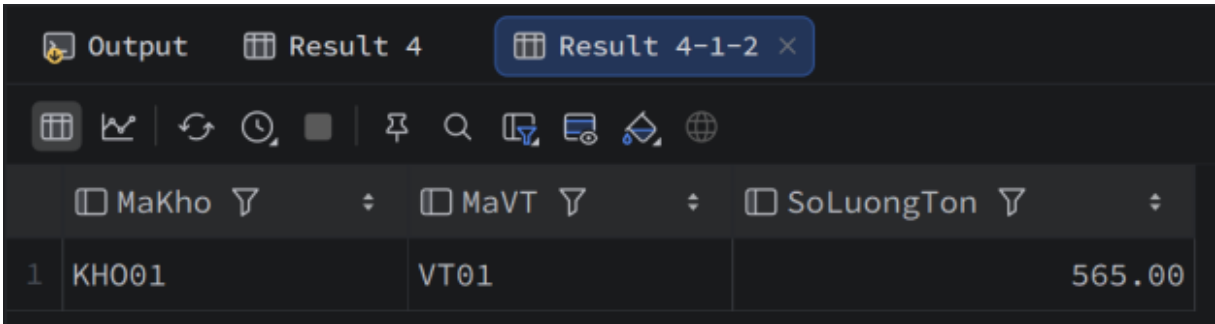


	MaKho	MaVT	SoLuongTon
1	KH001	VT01	465.00

Hình 3.x. Kết quả đọc dữ liệu lần thứ nhất của Session 1 với READ COMMITTED

Trong thời gian Session 1 đang tạm dừng bằng lệnh WAITFOR DELAY, Session 2 thực hiện cập nhật tăng số lượng tồn kho thêm 100 và COMMIT giao tác thành công.

Sau đó, Session 1 tiếp tục thực hiện câu lệnh SELECT lần thứ hai và kết quả nhận được là 565.00.



	MaKho	MaVT	SoLuongTon
1	KH001	VT01	565.00

Hình 3.x. Kết quả đọc dữ liệu lần thứ hai của Session 1 sau khi Session 2 cập nhật

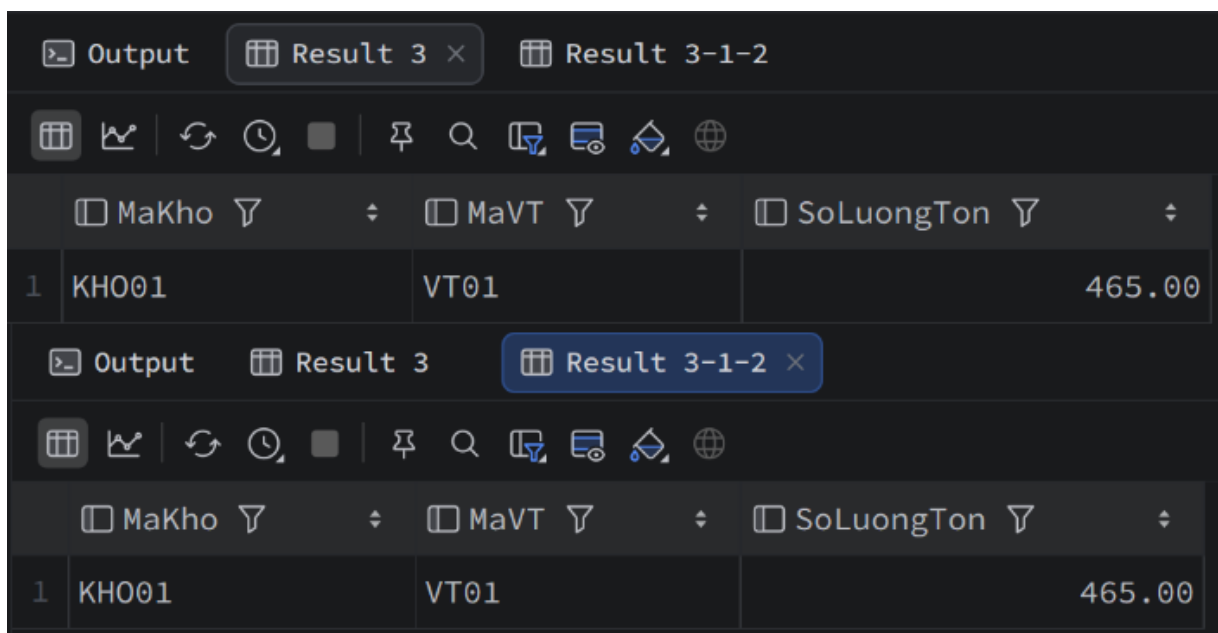
Kết quả cho thấy trong cùng một giao tác, hai lần đọc cùng một dữ liệu của Session 1 trả về hai giá trị khác nhau (465.00 và 565.00). Điều này chứng minh rằng hiện tượng **Non-repeatable Read** xảy ra ở mức cô lập **READ COMMITTED**.

Kết quả khi sử dụng REPEATABLE READ

Sau khi thay đổi mức cô lập sang REPEATABLE READ, Session 1 duy trì khóa đọc trên dữ liệu đã truy xuất trong suốt thời gian giao tác.

Trong thời gian Session 1 chưa thực hiện COMMIT, khi Session 2 cố gắng thực hiện lệnh UPDATE, thao tác cập nhật bị BLOCK và phải chờ Session 1 hoàn thành giao tác.

Sau khi Session 1 COMMIT, Session 2 mới có thể tiếp tục thực hiện cập nhật. Kết quả hai lần đọc dữ liệu của Session 1 đều trả về cùng một giá trị là 465.00



The image shows two screenshots of a database query result. Both screenshots display a table with three columns: MaKho, MaVT, and SoLuongTon. The data is as follows:

	MaKho	MaVT	SoLuongTon
1	KH001	VT01	465.00

The top screenshot shows the result of a query in Session 1, with tabs for 'Output', 'Result 3', and 'Result 3-1-2'. The bottom screenshot shows the same result in Session 2, with tabs for 'Output', 'Result 3', and 'Result 3-1-2'.

Hình 3.x. Kết quả khắc phục Non-repeatable Read bằng mức cô lập REPEATABLE READ

Kết quả trên cho thấy mức cô lập REPEATABLE READ đã ngăn chặn việc thay đổi dữ liệu giữa hai lần đọc trong cùng một giao tác, từ đó loại bỏ hiện tượng Non-repeatable Read. Tuy nhiên, việc duy trì khóa đọc lâu hơn làm giảm tính đồng thời (concurrency) của hệ thống do các giao tác ghi phải chờ. Vì vậy, mức cô lập này phù hợp với các nghiệp vụ yêu cầu dữ liệu nhất quán cao như kiểm tra và đối chiếu tồn kho trước khi xuất hàng.

e. Phân tích kết quả thực nghiệm

Kết quả thực nghiệm cho thấy mức cô lập READ COMMITTED chưa đảm bảo tính nhất quán của dữ liệu trong suốt vòng đời giao tác, do khóa đọc chỉ tồn tại trong thời gian thực hiện câu lệnh SELECT.

Khi sử dụng REPEATABLE READ, SQL Server duy trì khóa đọc đến khi giao tác kết thúc, ngăn các giao tác khác cập nhật dữ liệu đang được sử dụng. Nhờ đó, dữ liệu đọc trong cùng một giao tác luôn giữ nguyên và hiện tượng Non-repeatable Read được loại bỏ.

Tuy nhiên, việc giữ khóa trong thời gian dài làm giảm khả năng xử lý đồng thời của hệ thống. Vì vậy, mức cô lập này phù hợp với các nghiệp vụ yêu cầu độ chính xác cao như kiểm tra và đối chiếu tồn kho trước khi lập phiếu xuất kho.

Xử lý Deadlock

a. Mục đích thực nghiệm

Deadlock xảy ra khi hai giao tác cùng giữ khóa trên các tài nguyên khác nhau và đồng thời chờ khóa tài nguyên mà giao tác còn lại đang giữ. Khi đó, các giao tác tạo thành vòng chờ và không thể tiếp tục thực hiện.

Thực nghiệm được thực hiện nhằm mô phỏng lỗi Deadlock trong nghiệp vụ chuyển vật tư giữa hai kho, quan sát thông báo lỗi Msg 1205 - Deadlock Victim của SQL Server và áp dụng giải pháp Lock Ordering để ngăn chặn lỗi.

b. Kịch bản thực nghiệm

Thực nghiệm sử dụng bảng TON_KHO với vật tư VT01 tại hai kho KHO01 và KHO02, thực hiện trên **hai Session độc lập** trong SQL Server Management Studio.

Session 1: chuyển 5 vật tư từ KHO01 → KHO02, khóa KHO01 trước rồi yêu cầu khóa KHO02.

Session 2: chuyển 2 vật tư từ KHO02 → KHO01, khóa KHO02 trước rồi yêu cầu khóa KHO01.

Do hai giao tác khóa dữ liệu theo thứ tự ngược nhau nên hình thành vòng chờ (Circular Wait) và SQL Server phát hiện Deadlock.

c. Code thực nghiệm

SESSION 1: Chuyển VT01 từ KHO01 sang KHO02

```

1  USE QuanLyVatTuXayDung;
2  GO
3
4  BEGIN TRANSACTION;
5
6  PRINT N'[SESSION 1] Bat dau chuyen kho: KHO01 -> KHO02, VT01, SL = 5';
7
8  UPDATE TON_KHO
9  SET SoLuongTon = SoLuongTon - 5
10 WHERE MaKho = 'KHO01'
11      AND MaVT = 'VT01';
12
13 PRINT N'[SESSION 1] Da tru kho nguon KHO01. Dang giu khoa...';
14 WAITFOR DELAY '00:00:05';
15 PRINT N'[SESSION 1] Chuan bi cong kho dich KHO02...';
16
17 UPDATE TON_KHO
18 SET SoLuongTon = SoLuongTon + 5
19 WHERE MaKho = 'KHO02'
20      AND MaVT = 'VT01';
21
22 PRINT N'[SESSION 1] Thanh cong, tien hanh COMMIT.';
23
24 COMMIT TRANSACTION;
25 GO

```

SESSION 2: Chuyển VT01 từ KHO02 sang KHO01

```

1  USE QuanLyVatTuXayDung;
2  GO
3
4  BEGIN TRANSACTION;
5
6  PRINT N'[SESSION 2] Bat dau chuyen kho: KHO02 -> KHO01, VT01, SL = 2';
7
8  UPDATE TON_KHO
9  SET SoLuongTon = SoLuongTon - 2
10 WHERE MaKho = 'KHO02'
11      AND MaVT = 'VT01';
12
13 PRINT N'[SESSION 2] Da tru kho nguon KHO02. Dang giu khoa...';
14 WAITFOR DELAY '00:00:05';
15 PRINT N'[SESSION 2] Chuan bi cong kho dich KHO01...';
16
17 UPDATE TON_KHO
18 SET SoLuongTon = SoLuongTon + 2
19 WHERE MaKho = 'KHO01'
20      AND MaVT = 'VT01';
21
22 PRINT N'[SESSION 2] Thanh cong, tien hanh COMMIT.';
23
24 COMMIT TRANSACTION;
25 GO

```

c. Kết quả thực nghiệm

Sau khi chạy đồng thời hai Session, SQL Server phát hiện hai giao tác đang chờ khóa lẫn nhau và tự động chọn một giao tác làm Deadlock Victim.

SQL Server trả về thông báo lỗi Msg 1205:

```

7
8 UPDATE TON_KHO
9 SET SoLuongTon = SoLuongTon - 2
10 WHERE MaKho = 'KH002'
11 AND MaVT = 'VT01';
12
13 PRINT N'[SESSION 2] Da tru kho nguon KH002. Dang giu khoa...';
14 WAITFOR DELAY '00:00:05';
15 PRINT N'[SESSION 2] Chuan bi cong kho dich KH001...';
16
17 UPDATE TON_KHO
18 SET SoLuongTon = SoLuongTon + 2
19 WHERE MaKho = 'KH001'
20 AND MaVT = 'VT01';
21
22 PRINT N'[SESSION 2] Thanh cong, tien hanh COMMIT.';
23

```

Messages

[SESSION 2] Bat dau chuyen kho: KH002 -> KH001, VT01, SL = 2

(1 row affected)

[SESSION 2] Da tru kho nguon KH002. Dang giu khoa...

[SESSION 2] Chuan bi cong kho dich KH001...

Msg 1205, Level 13, State 51, Line 19

Transaction (Process ID 72) was deadlocked on lock resources with another process and has been chosen as the deadlock victim. Rerun the transaction.

Completion time: 2026-08-29T11:16:09.1146100+07:00

Hình 3.4.3.5.1. SQL Server phát hiện Deadlock và chọn một giao tác làm Deadlock Victim

```

1 USE QuanLyVatTuXayDung;
2 GO
3
4 BEGIN TRANSACTION;
5
6 PRINT N'[SESSION 1] Bat dau chuyen kho: KH001 -> KH002, VT01, SL = 5';
7
8 UPDATE TON_KHO
9 SET SoLuongTon = SoLuongTon - 5
10 WHERE MaKho = 'KH001'
11 AND MaVT = 'VT01';
12
13 PRINT N'[SESSION 1] Da tru kho nguon KH001. Dang giu khoa...';
14 WAITFOR DELAY '00:00:05';
15 PRINT N'[SESSION 1] Chuan bi cong kho dich KH002...';
16
17 UPDATE TON_KHO

```

Messages

[SESSION 1] Bat dau chuyen kho: KH001 -> KH002, VT01, SL = 5

(1 row affected)

[SESSION 1] Da tru kho nguon KH001. Dang giu khoa...

[SESSION 1] Chuan bi cong kho dich KH002...

(1 row affected)

[SESSION 1] Thanh cong, tien hanh COMMIT.

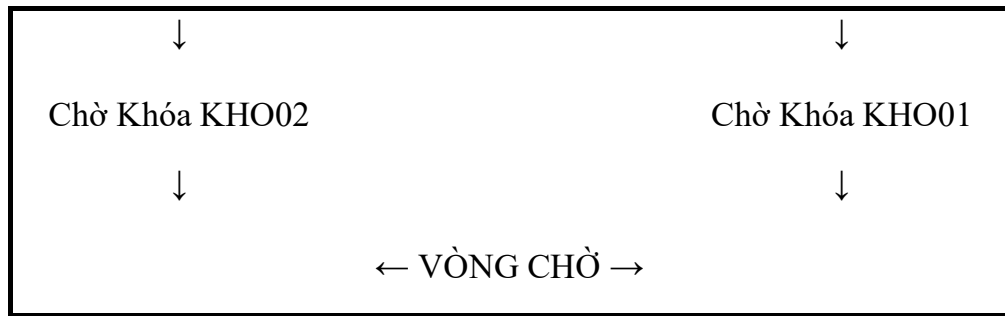
Completion time: 2026-08-29T11:16:09.1302307+07:00

Hình 3.4.3.5.2. Session còn lại tiếp tục thực hiện và COMMIT thành công

d. Phân tích lỗi

Nguyên nhân Deadlock được thể hiện như sau:

Sesion 1	Sesion 2
Giữ khóa KHO01	Giữ khóa KHO02



Session 1 đang giữ khóa tại KHO01 và chờ KHO02, trong khi Session 2 đang giữ khóa tại KHO02 và chờ KHO01. Hai giao tác tạo thành vòng chờ nên SQL Server phải hủy một giao tác để giải phóng tài nguyên.

SQL Server chọn một giao tác làm Deadlock Victim, giao tác này được rollback; giao tác còn lại tiếp tục thực hiện và COMMIT

e. Giải pháp - Lock Ordering

Để tránh Deadlock, áp dụng Lock Ordering - sắp xếp thứ tự khóa.

Nguyên tắc là tất cả các giao tác phải khóa các kho theo cùng một thứ tự, cụ thể:

KHO01 → KHO02

Do đó, ngay cả khi nghiệp vụ chuyển kho là KHO02 → KHO01, giao tác vẫn phải lấy khóa KHO01 trước.

Session 1 - Lock Ordering

```

1  USE QuanLyVatTuXayDung;
2  GO
3
4  BEGIN TRANSACTION;
5
6  PRINT N'[LOCK ORDERING - SESSION 1] Bat dau chuyen KHO01 -> KHO02';
7
8  -- Luôn khóa KHO01 trước
9  UPDATE TON_KHO
10 SET SoLuongTon = SoLuongTon - 5
11 WHERE MaKho = 'KHO01'
12    AND MaVT = 'VT01';
13
14 PRINT N'[SESSION 1] Da khoa KHO01. Dang giu khoa 5 giay...';
15
16 WAITFOR DELAY '00:00:05';
17
18 -- Sau đó mới khóa KHO02
19 UPDATE TON_KHO
20 SET SoLuongTon = SoLuongTon + 5
21 WHERE MaKho = 'KHO02'
22    AND MaVT = 'VT01';
23
24 PRINT N'[SESSION 1] Da cap nhat KHO02.';
25 PRINT N'[SESSION 1] COMMIT thanh cong.';
26
27 COMMIT TRANSACTION;
28 GO
29

```

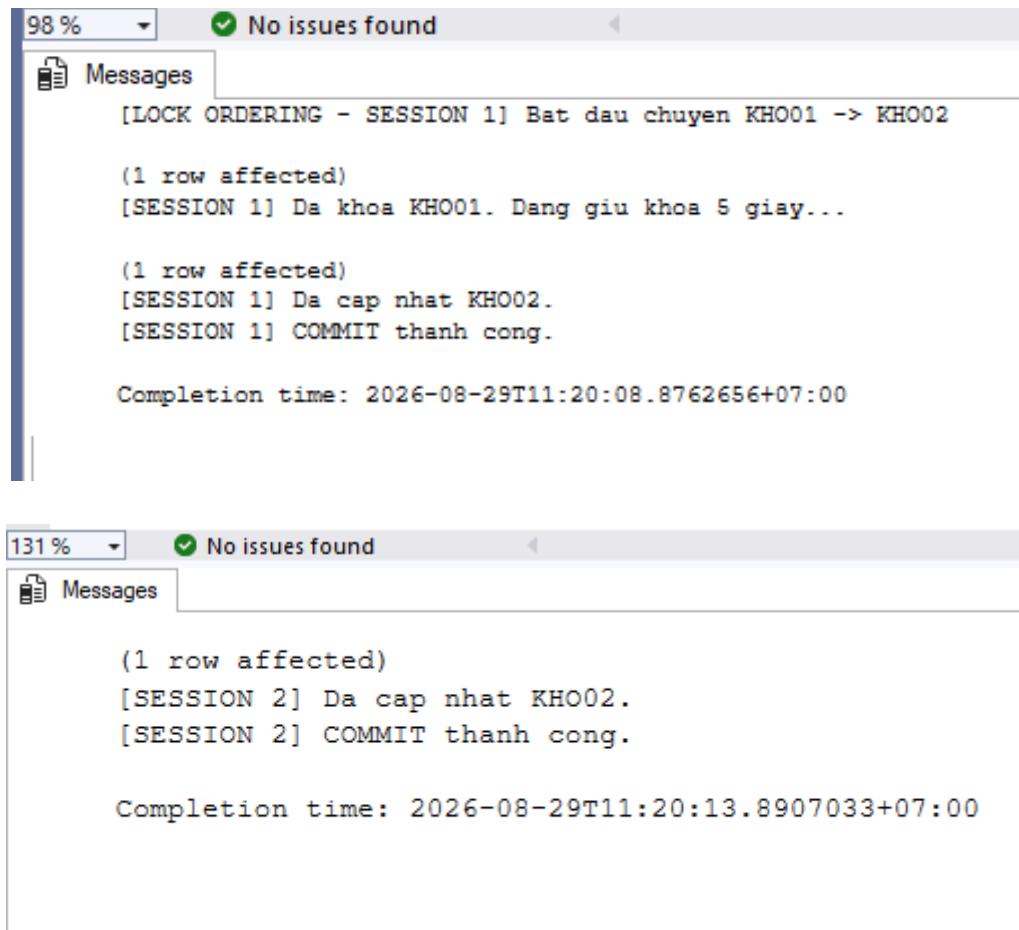
Session 2 - Lock Ordering

```

1  USE QuanLyVatTuXayDung;
2  GO
3
4  BEGIN TRANSACTION;
5
6  PRINT N'[LOCK ORDERING - SESSION 2] Bat dau chuyen KHO02 -> KHO01';
7
8  -- Dù chuyển từ KHO02 về KHO01,
9  -- vẫn phải khóa KHO01 trước
10 UPDATE TON_KHO
11 SET SoLuongTon = SoLuongTon + 2
12 WHERE MaKho = 'KHO01'
13    AND MaVT = 'VT01';
14
15 PRINT N'[SESSION 2] Da khoa KHO01. Dang giu khoa 5 giay...';
16
17 WAITFOR DELAY '00:00:05';
18
19 -- Sau đó mới khóa KHO02
20 UPDATE TON_KHO
21 SET SoLuongTon = SoLuongTon - 2
22 WHERE MaKho = 'KHO02'
23    AND MaVT = 'VT01';
24
25 PRINT N'[SESSION 2] Da cap nhat KHO02.';
26 PRINT N'[SESSION 2] COMMIT thanh cong.';
27
28 COMMIT TRANSACTION;
29 GO

```

Kết quả sau khi xử lý, cả hai Session đều COMMIT thành công và không xuất hiện Msg 1205.



f. Kết luận

Thực nghiệm cho thấy việc khóa các tài nguyên theo thứ tự khác nhau có thể tạo ra Deadlock và SQL Server sẽ chọn một giao tác làm Deadlock Victim với mã lỗi 1205.

Sau khi áp dụng Lock Ordering, cả hai giao tác đều thực hiện khóa theo cùng thứ tự KHO01 → KHO02. Vì vậy, có thể xảy ra hiện tượng một giao tác phải chờ, nhưng không hình thành vòng chờ, từ đó ngăn chặn Deadlock.

3.5. Các nội dung T-SQL bổ sung theo yêu cầu

CHƯƠNG 4: XÂY DỰNG VÀ ĐÁNH GIÁ CHƯƠNG TRÌNH

4.1. Công nghệ sử dụng và kết nối cơ sở dữ liệu

4.2. Các chức năng người dùng trên chương trình

4.3. Kết quả thực hiện

4.4. Kiểm thử và đánh giá hệ thống

CHƯƠNG 5: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1. Kết quả đạt được

5.2. Đánh giá ưu điểm, hạn chế của hệ thống

5.3. Kết luận kết quả đạt được

5.4. Đề xuất hướng phát triển

DANH MỤC TÀI LIỆU THAM KHẢO

MỨC ĐỘ GÓP CỦA THÀNH VIÊN TRONG NHÓM

TT	Họ và tên thành viên	Các nội dung, công việc thành viên đóng góp cho nhóm	Mức độ đóng góp
1			100%
2			100%
3			100%
4			100%
5			100%